

INFORMATIQUE • 1ÈRE TI

MODULE 2 : PROGRAMMATION

LEÇON N07

PROGRAMMATION JAVASCRIPT
CÔTÉ CLIENT

DURÉE CONSEILLÉE : 12 H

DANS CETTE LEÇON

Cette leçon introduit la programmation JavaScript côté client pour rendre les pages web dynamiques et interactives. Elle couvre l'insertion des scripts dans HTML, les variables, opérateurs, structures de contrôle, tableaux, fonctions, événements et le contrôle de saisie des formulaires.

- À la fin de cette leçon, je sais définir un script et un événement, et énoncer les caractéristiques, avantages et limites de JavaScript
- À la fin de cette leçon, je sais insérer un script JavaScript dans un document HTML (en-tête, corps, fichier externe)
- À la fin de cette leçon, je sais citer les types de base, déclarer des variables et des constantes
- À la fin de cette leçon, je sais utiliser les opérateurs arithmétiques, de comparaison, logiques, d'assignation et de concaténation
- À la fin de cette leçon, je sais utiliser les instructions simples (affectation, entrée, sortie) et les fonctions `parseInt()` et `parseFloat()`
- À la fin de cette leçon, je sais utiliser les structures alternatives (`if`, `if...else`) et les boucles (`for`, `while`, `do...while`)

TÉLÉCHARGE
ONBUCH

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats



PASSE À ONBUCH+

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à
pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir



Signé • OnBuch+

Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

1. Scripts, évènements et insertion de JavaScript dans HTML
2. Variables, constantes, types et opérateurs
3. Instructions simples : entrées, sorties et conversions
4. Structures alternatives et boucles
5. Tableaux, objets et fonctions
6. Évènements et contrôle des formulaires
7. Activité d'intégration
8. Méthodes et exercices résolus
9. Exercices
10. Corrigés des exercices
11. Fiche bilan

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais définir un script et un évènement, et énoncer les caractéristiques, avantages et limites de JavaScript
- À la fin de cette leçon, je sais insérer un script JavaScript dans un document HTML (entête, corps, fichier externe)
- À la fin de cette leçon, je sais citer les types de base, déclarer des variables et des constantes
- À la fin de cette leçon, je sais utiliser les opérateurs arithmétiques, de comparaison, logiques, d'assignation et de concaténation
- À la fin de cette leçon, je sais utiliser les instructions simples (affectation, entrée, sortie) et les fonctions `parseInt()` et `parseFloat()`
- À la fin de cette leçon, je sais utiliser les structures alternatives (`if`, `if...else`) et les boucles (`for`, `while`, `do...while`)
- À la fin de cette leçon, je sais déclarer, initialiser et parcourir un tableau, et créer un objet avec l'opérateur `new`
- À la fin de cette leçon, je sais écrire une fonction qui retourne une valeur ou non
- À la fin de cette leçon, je sais utiliser les évènements (`onClick`, `onMouseOver`, `onChange`, `onFocus`, `onLoad`) et contrôler la saisie d'un formulaire en accédant aux éléments HTML

PRÉREQUIS

- Structure d'un document HTML (balises head, body, form, input) vue en classe de Seconde et en début de Première
- Notion d'algorithme : variables, affectation, tests et boucles (module Algorithmique)
- Notion de fonction en mathématiques (paramètre, valeur de retour)
- Utilisation d'un navigateur web et d'un éditeur de texte (Notepad++, bloc-notes)

1 SCRIPTS, ÉVÈNEMENTS ET INSERTION DE JAVASCRIPT DANS HTML

Aminatou tient une boutique en ligne à Douala. Ses clientes remplissent un formulaire de commande, mais certaines oublient leur numéro de téléphone ou saisissent « abc » à la place de la quantité. Le site envoie alors des commandes incomplètes au serveur, ce qui fait perdre du temps et de l'argent : il faut rappeler les clientes, corriger les bons de livraison, annuler des envois. Un ami développeur lui explique qu'un petit programme exécuté directement dans le navigateur peut vérifier la saisie avant l'envoi, calculer le total automatiquement et réagir aux clics des visiteurs. Ce programme, c'est un **script** JavaScript.

Problématique : comment écrire un tel programme et comment l'insérer correctement dans les pages web d'Aminatou ? C'est toute l'ambition de cette première section : comprendre ce qu'est un script, ce qu'est un événement, connaître les forces et les faiblesses de JavaScript, puis maîtriser les trois façons d'insérer un script dans un document HTML.

1.1 Notion de script

1.2 Intuition : pourquoi un programme dans le navigateur ?

Jusqu'ici, avec HTML et CSS, tu as construit des pages statiques : belles, bien présentées, mais figées. Une page HTML seule ne peut ni calculer, ni vérifier une saisie, ni réagir à un clic. Or, dans la boutique d'Aminatou, on a besoin de logique : « si le champ téléphone est vide, afficher un avertissement ; sinon, envoyer la commande ». Cette logique, on l'écrit dans un langage de programmation. Mais où ce programme va-t-il s'exécuter ?

Deux réponses sont possibles. Soit le programme s'exécute sur le **serveur** (la machine distante qui héberge le site) : c'est la programmation côté serveur, que tu étudieras en Terminale. Soit il s'exécute sur la machine du visiteur, dans son **navigateur** : c'est la programmation côté client, et c'est précisément le rôle de **JavaScript**. L'intérêt est immense : la vérification se fait avant l'envoi, instantanément, sans aller-retour avec le serveur — un vrai soulagement quand la connexion est lente ou coûteuse.

DÉFINITION Script

Un **script** est un programme écrit dans un **langage interprété**, c'est-à-dire une suite d'instructions exécutées directement par un logiciel (ici, le navigateur web) sans compilation préalable. Contrairement à un programme en langage C, qui doit d'abord être traduit en langage machine par un compilateur, le script est lu et exécuté ligne par ligne, au moment où le navigateur le rencontre.

ATTENTION Interprété ne veut pas dire « sans règles »

Parce qu'il n'y a pas d'étape de compilation, une erreur de syntaxe dans un script n'est

détectée qu'à l'exécution, et souvent elle bloque silencieusement tout le reste du script. Il faut donc tester ses scripts dans le navigateur (la console de développement, touche F12, affiche les erreurs). Ne confonds pas non plus « interprété » et « lent » : les navigateurs modernes exécutent JavaScript très rapidement.

1.3 Notion d'évènement

Un script ne s'exécute pas toujours d'un seul coup au chargement de la page. Très souvent, il attend que quelque chose se passe : l'utilisateur clique sur un bouton, survole une image avec la souris, modifie un champ, ou la page finit de se charger. Chacune de ces actions est un **évènement**.

DÉFINITION Évènement

Un **évènement** est une action de l'utilisateur ou du navigateur (clic, survol de la souris, saisie dans un champ, chargement de la page, etc.) qui peut être détectée par le script et qui déclenche l'exécution d'un code JavaScript associé. On dit que le script « réagit » à l'évènement.

EXEMPLE Évènements dans la boutique d'Aminatou

- Clic sur le bouton « Commander » → évènement `onClick` : le script vérifie les champs.
- La cliente quitte le champ « quantité » après l'avoir modifié → évènement `onChange` : le script recalcule le total.
- La page vient de s'afficher → évènement `onLoad` : le script affiche un message de bienvenue.
- La cliente survole une image de produit → évènement `onMouseOver` : le script agrandit l'image.
- La cliente place le curseur dans le champ « email » → évènement `onFocus` : le script affiche une aide contextuelle.

Nous étudierons ces évènements en détail dans la dernière section de la leçon ; retiens pour l'instant l'idée générale : évènement détecté ⇒ code exécuté.

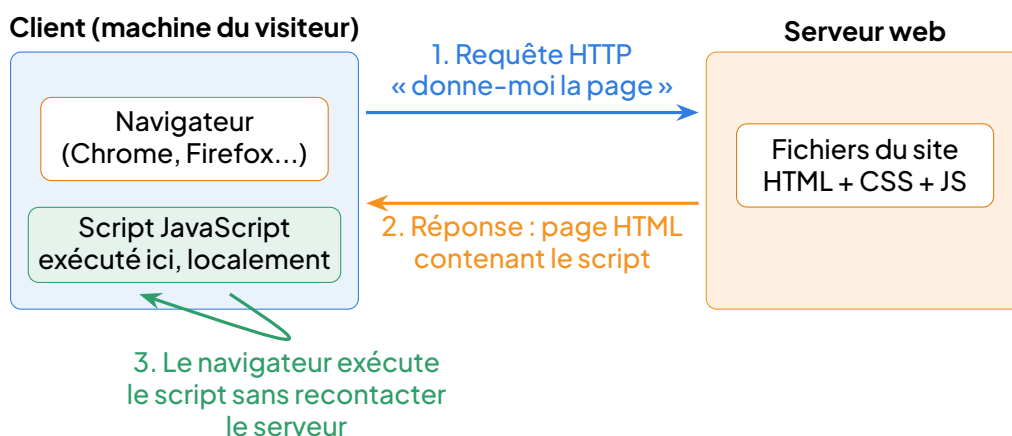
1.4 Caractéristiques, avantages et limites de JavaScript

1.5 Caractéristiques

PROPRIÉTÉ Caractéristiques du langage JavaScript

JavaScript est un langage :

- interprété : exécuté ligne par ligne par le navigateur, sans compilation préalable ;
- côté client : exécuté sur la machine du visiteur, dans son navigateur, et non sur le serveur ;
- orienté objet : il manipule des objets (la page, les balises, les champs de formulaire) dotés de propriétés et de méthodes ;
- faiblement typé : on ne déclare pas le type d'une variable ; le type est déterminé automatiquement à l'affectation et peut changer en cours d'exécution ;
- indépendant du serveur : il fonctionne même sans connexion au serveur, une fois la page chargée.



Architecture client/serveur : le serveur envoie la page et le script ; c'est le navigateur du client qui exécute le script localement.

1.6 Avantages

- **Interactivité immédiate** : la page réagit instantanément aux actions de l'utilisateur (calcul du total, message d'erreur, changement de couleur), sans attendre une réponse du serveur.
- **Allègement du serveur** : les traitements simples sont faits chez le client ; le serveur n'est sollicité que pour l'essentiel, ce qui est précieux quand des centaines de clientes commandent en même temps.
- **Contrôle de saisie avant envoi** : on vérifie les formulaires (champ vide, quantité non numérique) avant de transmettre les données — c'est exactement le problème d'Aminatou.
- **Amélioration de l'IHM** : menus déroulants, messages de confirmation, effets visuels rendent l'interface homme-machine plus agréable et plus claire.

1.7 Limites

ATTENTION Limites à connaître

- JavaScript dépend du navigateur : un script peut se comporter légèrement différemment selon le navigateur ou sa version.
- Le code est visible par l'utilisateur (affichage du code source) : on n'y place donc jamais de secret (mot de passe, clé) ; la sécurité est limitée.
- JavaScript peut être désactivé par l'utilisateur dans les réglages du navigateur : le site doit rester utilisable, ou prévoir un message d'avertissement.
- Pour des raisons de sécurité, JavaScript n'a pas accès aux fichiers de la machine du client : il ne peut ni lire ni écrire librement sur le disque dur du visiteur.

Conséquence pratique : le contrôle JavaScript est un confort, jamais une garantie absolue ; les vérifications critiques seront refaites côté serveur (en Terminale).

LESAVAIS-TU ? JavaScript n'a rien à voir avec Java

Malgré son nom, JavaScript n'a aucun lien avec le langage Java. Créé en 1995 par Brendan Eich chez Netscape, il s'appelait d'abord Mocha, puis LiveScript. Le nom « JavaScript » fut choisi pour profiter de l'énorme popularité de Java à l'époque — un coup de marketing qui sème la confusion chez les débutants depuis près de trente ans ! Son nom officiel standardisé est d'ailleurs ECMAScript.

1.8 Insertion d'un script dans un document HTML

Tout script JavaScript placé dans une page HTML est délimité par la balise `<script>` et sa balise fermante `</script>`. Reste à savoir où la placer : il existe trois emplacements possibles, chacun avec son usage.

PROPRIÉTÉ Syntaxe générale de la balise script

Script interne (code écrit dans la page) :

```
<script>  
  // instructions JavaScript  
</script>
```

Script externe (code dans un fichier séparé) :

```
<script src="monscript.js"></script>
```

L'attribut `src` (source) indique le chemin du fichier externe, de la même façon que pour une image.

1.9 Insertion dans l'en-tête

Le script est placé entre `<head>` et `</head>`. Il est chargé avant le contenu de la page. C'est l'emplacement idéal pour les **fonctions réutilisables** qui seront appelées plus tard, par exemple au clic sur un bouton : elles sont déjà en mémoire quand l'utilisateur agit.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Boutique d'Aminatou</title>
  <script>
    // Fonction définie dans l'en-tête, prête à l'emploi
    function saluer() {
      alert("Bienvenue dans ma boutique !");
    }
  </script>
</head>
<body>
  <h1>Boutique d'Aminatou</h1>
</body>
</html>
```

1.10 Insertion dans le corps

Le script est placé dans `<body>`, à l'endroit exact où l'on veut qu'il s'exécute. Il est exécuté **immédiatement**, au moment où le navigateur le rencontre pendant l'affichage de la page. C'est l'emplacement adapté à une exécution immédiate, par exemple écrire du texte dans la page.

```
<body>
  <h1>Nos produits</h1>
  <script>
    // Exécute pendant l'affichage de la page
    document.write("<p>Promotions du jour !</p>");
  </script>
  <p>Livraison partout à Douala.</p>
</body>
</html>
```

1.11 Insertion d'un fichier externe

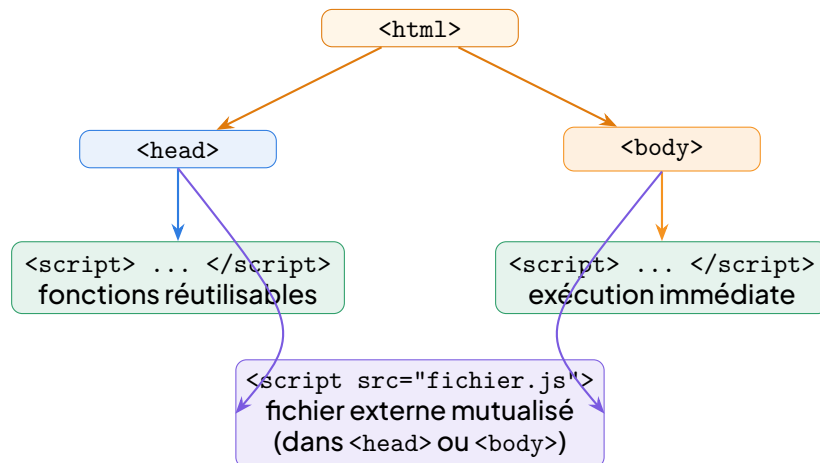
Le code est écrit dans un fichier séparé, par exemple `boutique.js`, et la page y fait référence grâce à l'attribut `src`. C'est la méthode **professionnelle** : le même script peut servir dans dix pages (réutilisabilité), et le code JavaScript reste séparé du code HTML, ce qui rend le site plus facile à maintenir.

Fichier `boutique.js` :

```
// Fichier externe : pas de balise <script> ici !  
function verifierQuantite() {  
    alert("Vérification de la quantité...");  
}
```

Page HTML qui l'utilise :

```
<head>  
  <meta charset="utf-8">  
  <title>Commande</title>  
  <script src="boutique.js"></script>  
</head>
```



Arborescence d'un document HTML : les trois emplacements possibles de la balise `<script>`.

MÉTHODE Choisir l'emplacement du script

1. Le script contient des fonctions appelées par des événements (clics, survols)? → place-le dans l'en-tête (`<head>`), afin qu'elles soient chargées avant toute action de l'utilisateur.
2. Le script doit s'exécuter au moment de l'affichage (écrire dans la page, message de bienvenue)? → place-le dans le corps (`<body>`), à l'endroit voulu.
3. Le même script sert dans plusieurs pages? → crée un fichier externe `.js` et lie-le avec `src` dans chaque page.

ATTENTION Erreurs fréquentes avec la balise script

- Oublier la balise fermante `</script>` : tout le contenu qui suit est alors considéré comme du JavaScript et la page ne s'affiche plus correctement.
- Écrire `scr` au lieu de `src` dans `<script src="...">` : le fichier externe n'est jamais chargé, et aucune erreur visible ne s'affiche — le script semble « ignoré ».
- Mettre des balises `<script>` à l'intérieur du fichier `.js` : le fichier externe ne contient que du code JavaScript pur.
- Oublier que JavaScript est sensible à la casse : `Alert` n'existe pas, seul `alert` fonctionne.
- Utiliser `document.write()` après le chargement de la page (dans un gestionnaire d'évènement par exemple) : cela efface tout le contenu de la page pour la réécrire. Réserve `document.write()` aux scripts s'exécutant pendant l'analyse initiale du HTML.

1.12 Premier script : alert() et document.write()

Deux instructions de sortie suffisent pour débiter :

- `alert("message")` affiche une **boîte de dialogue** avec un bouton OK ; elle bloque la page jusqu'au clic de l'utilisateur. Idéale pour les avertissements (« Champ téléphone vide ! »).
- `document.write("texte")` **écrit directement dans la page HTML**, à l'endroit où se trouve le script (uniquement pendant le chargement initial).

EXPÉRIENCE TP guidé : mon premier script

Objectif : afficher un message de bienvenue et écrire une ligne dans la page d'accueil de la boutique.

Matériel : un ordinateur avec un éditeur de texte (Notepad++, VS Code) et un navigateur (Chrome, Firefox).

Manipulation :

1. Ouvre l'éditeur et saisis le code suivant :

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Premier script</title>
  <script>
    // Boîte de dialogue affichée au chargement
    alert("Bienvenue chez Aminatou !");
  </script>
</head>
```

```
<body>
  <h1>Ma boutique en ligne</h1>
  <script>
    // Écriture directe dans la page
    document.write("<p>Ouvert du lundi au samedi.</p>");
  </script>
</body>
</html>
```

2. Enregistre le fichier sous le nom `index.html` (type « Tous les fichiers », encodage UTF-8).
3. Double-clique sur le fichier : il s'ouvre dans le navigateur.

Observations : une boîte de dialogue « Bienvenue chez Aminatou! » apparaît d'abord (script de l'en-tête); après le clic sur OK, la page s'affiche avec le titre et la ligne « Ouvert du lundi au samedi. » (script du corps).

Interprétation : le navigateur a exécuté les deux scripts localement, dans l'ordre où il les a rencontrés, sans aucun serveur — la preuve : le fichier fonctionne même hors connexion.

EXERCICE RÉSOLU 1**Insertion correcte d'un fichier externe**

Énoncé : Aminatou a écrit le code ci-dessous, mais le message ne s'affiche jamais. Trouve les deux erreurs et corrige-les.

```
<head>
  <script scr="accueil.js">
</head>
```

SOLUTION

Solution détaillée :

1. Première erreur : l'attribut est mal orthographié : `scr` au lieu de `src`. Le navigateur ne trouve donc aucun fichier à charger.
2. Deuxième erreur : la balise fermante `</script>` est absente. Même pour un fichier externe (dont le corps est vide), la balise fermante est obligatoire.

Code corrigé :

```
<head>
  <script src="accueil.js"></script>
</head>
```

Le fichier `accueil.js` doit se trouver dans le même dossier que la page HTML, sinon il faut indiquer le chemin complet (par exemple `src="js/accueil.js"`).

EXERCICE 1 À toi de jouer

1. Donne deux avantages et deux limites de JavaScript, en les illustrant par la situation de la boutique d'Aminatou.
2. Écris une page HTML complète dont le script, placé dans le corps, affiche avec `document.write()` la phrase : « Bienvenue sur le site du Lycée de Nkol-Eton ».
3. On souhaite qu'une fonction `verifier()` soit disponible dès le chargement de la page pour être appelée au clic sur un bouton. Où faut-il placer le script et pourquoi ?

CORRIGÉ Exercice

1. Avantages : contrôle de saisie avant envoi (évite les commandes sans téléphone) et interactivité immédiate (total calculé sans attendre le serveur). Limites : le code est visible par les visiteurs (pas de secret commercial dans le script) et JavaScript peut être désactivé (le formulaire doit alors être vérifié autrement).

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Lycée</title>
</head>
<body>
  <script>
    document.write("Bienvenue sur le site du Lycée de Nkol-Eton");
  </script>
</body>
</html>
```

3. Dans l'en-tête (`<head>`) : la fonction `y` est chargée avant l'affichage du corps, donc elle est disponible en mémoire dès que l'utilisateur clique sur le bouton.

À RETENIR L'essentiel de la section

- Un **script** est un programme en langage interprété, exécuté directement par le navigateur sans compilation.
- Un **évènement** (clic, survol, chargement...) déclenche l'exécution d'un code JavaScript.
- JavaScript est interprété, côté client, orienté objet, faiblement typé et indépendant du serveur ; il rend les pages interactives mais reste visible, désactivable et sans accès

aux fichiers du client.

- Trois insertions possibles : en-tête (fonctions réutilisables), corps (exécution immédiate), fichier externe via `<script src="fichier.js"></script>` (mutualisation).
- `alert()` affiche une boîte de dialogue ; `document.write()` écrit dans la page (pendant le chargement).

2 VARIABLES, CONSTANTES, TYPES ET OPÉRATEURS

Dans la section précédente, nous avons appris à insérer un script JavaScript dans une page HTML et à le déclencher grâce à un événement comme `onClick`. Mais un script qui ne fait qu'afficher un message fixe reste limité. Un vrai programme doit pouvoir mémoriser des informations : le prix d'un article dans la boutique en ligne, la note d'un élève, le nom du visiteur. C'est précisément le rôle des **variables**. Et pour transformer ces informations (calculer une moyenne, comparer deux montants, décider si un élève est admis), il faut des **opérateurs**. Cette section pose donc les fondations de tout programme JavaScript.

2.1 Variables et constantes

Intuition. Imagine une étagère de boîtes étiquetées dans un magasin de Douala : chaque boîte porte un nom (son étiquette) et contient une valeur (son contenu). Tu peux ouvrir la boîte pour lire le contenu ou le remplacer. En informatique, ces boîtes sont des cases de la mémoire vive de l'ordinateur.

DÉFINITION Variable et constante

Une **variable** est une zone de la mémoire nommée (on lui donne un identificateur) dont le contenu peut varier au cours de l'exécution du programme. Une **constante** est une zone de la mémoire nommée dont le contenu est fixé une fois pour toutes à la déclaration : toute tentative de modification ultérieure provoque une erreur.

Déclaration. En JavaScript, on déclare une variable avec le mot-clé `var` (programme officiel), suivi du nom, puis on lui affecte une valeur avec `=`. On peut déclarer et initialiser en une seule instruction. Le mot-clé `let`, introduit plus tard dans le langage, joue le même rôle que `var` avec une portée plus fine (complément utile pour le Bac). Les constantes se déclarent avec `const` et doivent être initialisées immédiatement.

```
var nom = "Amina";    // variable chaine de caracteres
var age = 16;         // variable numerique
var note;             // declaree mais pas encore initialisee
note = 14.5;          // affectation ulterieure
let ville = "Yaounde"; // equivalent moderne de var
const PI = 3.14159;    // constante : valeur definitive
const TAUX_TVA = 0.1925; // TVA du Cameroun : 19,25 %
```

```
// PI = 3.14;           // ERREUR : une constante ne se modifie pas
```

PROPRIÉTÉ Règles de nommage des identificateurs

Le nom d'une variable ou d'une constante en JavaScript obéit aux règles suivantes :

- il est composé uniquement de lettres (a-z, A-Z), de chiffres (0-9) et du caractère souligné `_` ;
- il ne doit jamais commencer par un chiffre ;
- JavaScript est sensible à la casse : `note`, `Note` et `NOTE` sont trois variables différentes ;
- les mots réservés du langage (`var`, `if`, `for`, `function`...) sont interdits.

Par convention, on écrit les noms en minuscules avec une majuscule à chaque nouveau mot : `prixUnitaire`, `nombreEleves`.

ATTENTION Noms invalides

`2note` est refusé (commence par un chiffre), `prix-unitaire` est refusé (le tiret est interprété comme une soustraction), `ma note` est refusé (espace interdit). Et surtout : si tu declares `Moyenne` puis écris `moyenne` plus loin, JavaScript ne signalera pas toujours l'erreur clairement et ton programme se comportera bizarrement. Vérifie l'orthographe exacte de tes identificateurs !

2.2 Les types de base en JavaScript

Chaque valeur manipulée possède un **type**. Le programme officiel en retient trois :

- **number** : les nombres, entiers (42, -7) ou réels (14,5, 0, 1925). Attention : le séparateur décimal est le **point**, jamais la virgule ;
- **string** : les chaînes de caractères, délimitées par des guillemets doubles ou simples : `"Bonjour"`, `'Cameroun'` ;
- **boolean** : les valeurs logiques, qui ne peuvent être que `true` (vrai) ou `false` (faux).

PROPRIÉTÉ Typage faible et dynamique (admise)

JavaScript est un langage faiblement typé : on ne déclare jamais le type d'une variable. Le type est déterminé automatiquement au moment de l'affectation, et il peut changer au cours du programme. Ainsi, après `var x = 5` ; la variable `x` est un `number` ; après `x = "cinq"` ; elle devient une `string`. C'est souple, mais source d'erreurs : il faut toujours savoir ce que contient une variable avant de l'utiliser.

```
var x = 5;           // x est de type number
x = "cinq";         // x devient de type string (autorise !)
var admis = true;    // admis est de type boolean
```

2.3 Les opérateurs arithmétiques

Les opérateurs arithmétiques réalisent les calculs sur les nombres. Ce sont : + (addition), - (soustraction), * (multiplication), / (division) et % (**modulo** : reste de la division entière).

EXEMPLE Le modulo, l'opérateur méconnu

Le modulo `a % b` donne le reste de la division entière de `a` par `b`. Il est très utilisé, par exemple pour tester la parité d'un nombre.

```
var a = 5, b = 2;
var q = a / b;    // q vaut 2.5 (division réelle)
var r = a % b;    // r vaut 1 (reste de 5 divisé par 2)
var p = 14 % 2;   // p vaut 0 : 14 est pair
var i = 15 % 2;   // i vaut 1 : 15 est impair
```

2.4 L'opérateur d'assignation et ses formes combinées

L'opérateur `=` ne signifie pas « est égal à » mais « reçoit » : l'instruction `x = x + 1`; se lit « `x` reçoit l'ancienne valeur de `x` augmentée de 1 ». Le membre de droite est évalué d'abord, puis rangé dans la variable de gauche.

Comme on modifie très souvent une variable à partir de sa propre valeur, JavaScript offre des formes combinées : `+=`, `-=`, `*=`, `/=`, ainsi que `++` (incrément : ajouter 1) et `--` (décrément : retirer 1).

MÉTHODE Accumuler une valeur

Pour accumuler des montants (total d'une facture, somme de notes), la démarche est toujours la même :

1. initialiser l'accumulateur à zéro : `var total = 0`;
2. à chaque nouvelle valeur, écrire `total += prix`; qui est strictement équivalent à `total = total + prix`;
3. lire `total` à la fin.

```
var total = 0;
total += 1500; // total vaut 1500 (savon)
total += 2500; // total vaut 4000 (huile)
total += 1000; // total vaut 5000 (riz)
var compteur = 0;
compteur++;    // compteur vaut 1
```

```
compteur++; // compteur vaut 2
```

2.5 Les opérateurs de comparaison

Les opérateurs de comparaison comparent deux valeurs et produisent un **booléen** (`true` ou `false`) : `==` (égal), `!=` (différent), `<` (strictement inférieur), `>` (strictement supérieur), `<=` (inférieur ou égal), `>=` (supérieur ou égal). On mentionne aussi `===` (égalité stricte, qui compare en plus le type) : `5 == "5"` vaut `true` (conversion automatique) alors que `5 === "5"` vaut `false` (complément utile pour le Bac).

ATTENTION Le piège classique : `=` contre `==`

Dans un test, confondre `=` (affectation) et `==` (comparaison) est l'erreur la plus fréquente du débutant. Écrire `if (note = 10)` ne compare pas : cela affecte 10 à `note`, et le test est alors toujours vrai ! Pour comparer, il faut impérativement le double égal : `if (note == 10)`.

2.6 Les opérateurs logiques

Les opérateurs logiques combinent des conditions booléennes : `&&` (ET logique), `||` (OU logique), `!` (NON logique). Par exemple, un élève est admis au Probatoire si sa moyenne est supérieure ou égale à 10 et sa conduite jugée satisfaisante : `(moyenne >= 10) && (conduite == true)`.

ET : `a && b`

a	b	résultat
false	false	false
false	true	false
true	false	false
true	true	true

OU : `a || b`

a	b	résultat
false	false	false
false	true	true
true	false	true
true	true	true

NON : `!a`

a	résultat
false	true
true	false

Tables de vérité des opérateurs logiques : le ET n'est vrai que si les deux conditions sont vraies ; le OU est vrai dès qu'au moins une condition est vraie ; le NON inverse la valeur.

2.7 La concaténation et le piège de l'opérateur +

L'opérateur `+` a un comportement double : avec deux nombres, il **additionne** ; dès que l'un des deux opérandes est une chaîne, il **concatène** (colle les textes bout à bout).

EXEMPLE Addition ou concaténation ?

```

var nom = "Amina";
var message = "Bonjour" + nom; // "Bonjour Amina" (concatenation)
var x = 5 + 2;                 // 7 (addition : deux nombres)
var y = "5" + 2;               // "52" (concatenation : "5" est une chaîne)
var z = "5" + 2 + 3;           // "523" (de gauche à droite)
var w = 5 + 2 + "3";           // "73" (5+2=7 d'abord, puis collage)

```

ATTENTION Le piège du + avec les saisies

Les valeurs lues dans un formulaire HTML sont toujours des chaînes, même si l'utilisateur tape des chiffres. Ainsi "5000" + "1500" donne "50001500" et non 6500 ! Nous verrons dans la section suivante comment convertir avec `parseInt()` et `parseFloat()` avant tout calcul.

Catégorie	Opérateurs	Exemple	Résultat
Arithmétiques	+ - * / %	5 % 2	1
Comparaison	== != < > <= >=	7 >= 10	false
Logiques	&& !	true && false	false
Assignation	= += -= *= /= ++	t=3; t += 4;	t vaut 7
Concaténation	+ (avec une chaîne)	"Note : " + 14	"Note : 14"

Tableau récapitulatif des opérateurs JavaScript par catégorie, avec un exemple et son résultat pour chacune.

À RETENIR L'essentiel de la section

- Une variable se déclare avec `var` (ou `let`), une constante avec `const`; les noms respectent des règles strictes et la casse compte.
- Les trois types de base sont `number`, `string` et `boolean`; JavaScript est faiblement typé : le type suit la valeur affectée.
- `%` donne le reste de la division entière; `==` compare, `=` affecte; `&&`, `||`, `!` combinent les conditions.

- `total += prix;` est la forme courte de `total = total + prix;`.
- `+` additionne deux nombres mais concatène dès qu'une chaîne est présente : `"5" + 2` donne `"52"`.

EXERCICE RÉSOLU 2 Facture d'une boutique scolaire

Énoncé. Une librairie de Bafoussam vend 3 cahiers à 500 FCFA l'unité et 2 stylos à 150 FCFA l'unité. Écrire les instructions JavaScript qui calculent le montant total, puis déterminent si l'achat dépasse 2000 FCFA (booléen `dépassement`), et affichent un message du type « Total : 1800 FCFA ». Que vaut `total % 1000`?

SOLUTION

Solution détaillée. On déclare les quantités et prix unitaires, on calcule le total par accumulation, puis on compare.

```
var nbCahiers = 3, prixCahier = 500;
var nbStylos = 2, prixStylo = 150;
var total = 0;
total += nbCahiers * prixCahier; // total = 1500
total += nbStylos * prixStylo; // total = 1800
var depassement = total > 2000; // false : 1800 n'excede pas 2000
var message = "Total : " + total + " FCFA";
// message vaut "Total : 1800 FCFA" (concatenation)
var reste = total % 1000; // reste de 1800 / 1000 : 800
```

Vérifications : $3 \times 500 = 1500$, $2 \times 150 = 300$, $1500 + 300 = 1800$. La comparaison $1800 > 2000$ est fausse, donc `dépassement` vaut `false`. Enfin, $1800 = 1 \times 1000 + 800$, donc `total % 1000` vaut 800. Remarque le mélange dans `message` : `"Total : "` étant une chaîne, le `+` concatène le nombre 1800 au lieu de l'additionner.

EXERCICE 2 À toi de jouer

1. Déclarer une constante `SEUIL` égale à 10 et une variable `moyenne` égale à 12.5, puis écrire l'expression booléenne qui teste si l'élève est admis.
2. Que valent `17 % 5`, `"7" + "3"`, `7 + "3"` et `!(4 > 2) || (1 == 2)`? Justifier chaque réponse.
3. Réécrire `s = s - 8`; et `n = n + 1`; avec les formes combinées.

CORRIGÉ Exercice : À toi de jouer

1. `const SEUIL = 10; var moyenne = 12.5; puis le test : moyenne >= SEUIL qui vaut true.`
2. `17 % 5` vaut 2 car `17 = 3 × 5 + 2`; `"7" + "3"` vaut `"73"` (deux chaînes : concaténation); `7 + "3"` vaut `"73"` (une chaîne suffit à déclencher la concaténation); `!(4 > 2) || (1 == 2)` vaut `false` || `false`, donc `false` : en effet `4 > 2` est vrai, son NON est faux, et `1 == 2` est faux.
3. `s -= 8; et n++; (ou n += 1;).`

3 INSTRUCTIONS SIMPLES : ENTRÉES, SORTIES ET CONVERSIONS

Dans la section précédente, tu as appris à déclarer des variables et des constantes, à choisir leur type et à manipuler les opérateurs. Mais un programme qui ne fait que calculer en silence, avec des valeurs écrites « en dur » dans le code, ne sert à rien. Imagine le site d'une boutique de Douala : le client doit pouvoir saisir la quantité d'articles qu'il commande, et la page doit lui afficher le montant total. Toute application interactive repose donc sur trois gestes fondamentaux : **lire** une donnée (entrée), la **transformer** (calcul), puis la **restituer** (sortie). Cette section t'apprend à réaliser ces trois gestes en JavaScript, et surtout à éviter le piège le plus célèbre du débutant : la confusion entre chaîne de caractères et nombre.

3.1 L'affectation : stocker le résultat d'une expression

DÉFINITION Affectation

L'**affectation** est l'instruction qui consiste à évaluer une expression (à droite du signe =) puis à stocker le résultat dans une variable (à gauche du signe =). En JavaScript, sa syntaxe est :

```
variable = expression ;
```

L'ordre des opérations est capital et toujours le même, dans tous les langages de programmation :

1. le moteur JavaScript **évalue** d'abord l'expression située à droite du = (calculs, appels de fonctions, concaténations...);
2. le résultat obtenu est ensuite **rangé** dans la variable située à gauche, en écrasant son ancienne valeur.

EXEMPLE Lecture d'affectations

```
let prixUnitaire = 1500; // on stocke 1500
let quantite = 3; // on stocke 3
let total = prixUnitaire * quantite; // on évalue 1500*3, puis on stocke 4500
total = total + 500; // on évalue 4500+500, puis on stocke 5000
```

La dernière ligne n'est pas une équation mathématique : elle signifie « prends la valeur actuelle de `total`, ajoute 500, et remets le résultat dans `total` ». Après exécution, `total` vaut 5000.

ATTENTION Le signe `=` n'est pas une égalité mathématique

En JavaScript, `=` est l'opérateur d'affectation, pas de comparaison. Écrire `total = 500` modifie la variable ; pour tester une égalité, on utilise `==` ou `===` (vu dans la section précédente). Écrire `if (total = 500)` au lieu de `if (total === 500)` est un bug classique : la condition sera toujours vraie et la variable sera écrasée !

3.2 Les sorties : communiquer un résultat à l'utilisateur

Un programme qui calcule sans rien montrer est inutile. JavaScript côté client offre trois moyens principaux de sortie, chacun ayant son usage propre.

1. La boîte de dialogue `alert()`.

PROPRIÉTÉ Syntaxe de `alert()`

```
alert(message);
```

`alert()` ouvre une boîte de dialogue modale (qui bloque la page tant que l'utilisateur n'a pas cliqué sur « OK ») affichant le `message`. Le message peut être une chaîne littérale, une variable, ou une concaténation des deux.

```
let nom = "Amina";
alert("Bonjour " + nom + ", bienvenue sur notre site !");
// Affiche : Bonjour Amina, bienvenue sur notre site !
```

2. L'écriture dans la page avec `document.write()`.

PROPRIÉTÉ Syntaxe de `document.write()`

```
document.write(texte);
```

`document.write()` écrit le texte directement dans le corps du document HTML, à l'endroit où le script est exécuté. Le texte peut contenir des balises HTML, qui seront interprétées par le navigateur.

```
let moyenne = 14,5;
document.write("Votre moyenne est : " + moyenne + " / 20");
document.write("<br>"); // <br> : saut de ligne HTML
document.write("<strong>Admis(e) !</strong>"); // balises interprétées
```

ATTENTION `document.write()` après le chargement de la page

Si `document.write()` est appelé après que la page a fini de se charger (par exemple dans une fonction déclenchée par un clic), il efface tout le contenu existant de la page avant d'écrire. On l'utilisera donc uniquement pendant le chargement initial, ou à des fins de test. Pour modifier une page déjà affichée, on préférera `getElementById()` (vu en section 6).

3. Le débogage avec `console.log()`.

PROPRIÉTÉ Syntaxe de `console.log()`

```
console.log(valeur);
```

`console.log()` affiche la valeur dans la console du navigateur (touche F12, onglet « Console »), invisible pour le visiteur. C'est l'outil préféré du programmeur pour vérifier le contenu d'une variable en cours d'exécution et traquer les bugs.

```
let a = 7, b = 3;
console.log("a =", a, " b =", b, " a/b =", a / b);
// Dans la console : a = 7  b = 3  a/b = 2,3333333333333335
```

3.3 L'entrée : lire une donnée avec `prompt()`

DÉFINITION La fonction `prompt()`

`prompt()` ouvre une boîte de dialogue contenant un message et un champ de saisie, puis attend que l'utilisateur tape une valeur et valide. Sa syntaxe est :

```
variable = prompt(message, valeurParDefaut);
```

- **message** : le texte affiché pour guider l'utilisateur ;
- **valeurParDefaut (facultatif)** : le texte pré-rempli dans le champ de saisie ;
- la fonction retourne ce que l'utilisateur a tapé, que l'on stocke dans la `variable`.

EXEMPLE Première saisie

```
let ville = prompt("Dans quelle ville habites-tu ?", "Yaoundé");  
alert("Tu habites à " + ville + ".");
```

Si l'utilisateur valide sans rien changer, `ville` contient "Yaoundé". S'il tape « Bafoussam », `ville` contient "Bafoussam".

À RETENIR La règle d'or de `prompt()`

`prompt()` retourne toujours une chaîne de caractères (type `string`), même si l'utilisateur tape des chiffres. Taper 2000 dans la boîte de dialogue produit la chaîne "2000", pas le nombre 2000. Si l'utilisateur clique sur « Annuler », `prompt()` retourne la valeur spéciale `null`.

3.4 La conversion : `parseInt()` et `parseFloat()`

Voici le problème central de cette section. Puisque `prompt()` renvoie une chaîne, l'opérateur `+` appliqué à deux saisies réalise une **concaténation** et non une addition :

```
let a = prompt("Premier nombre :"); // l'utilisateur tape 2000  
let b = prompt("Deuxième nombre :"); // l'utilisateur tape 500  
alert(a + b); // Affiche "2000500" et NON 2500 !
```

Pour faire de vrais calculs, il faut donc **convertir** la chaîne saisie en nombre. JavaScript fournit deux fonctions prédéfinies pour cela.

PROPRIÉTÉ Syntaxe de `parseInt()` et `parseFloat()`

```
nombreEntier = parseInt(chaine); // conversion en entier  
nombreReel = parseFloat(chaine); // conversion en nombre réel
```

- `parseInt(chaine)` lit les chiffres au début de la chaîne et retourne un nombre entier : la

partie décimale est ignorée (tronquée, pas arrondie) ;

- `parseFloat(chaine)` lit les chiffres et le point décimal et retourne un nombre réel ;
- si la chaîne ne commence par aucun chiffre valide, les deux fonctions retournent la valeur spéciale **NaN** (Not a Number).

EXEMPLE Exemples chiffrés de conversion

```
parseInt("12.7"); // vaut 12 (partie décimale tronquée)
parseFloat("12.7"); // vaut 12,7 (réel conservé)
parseInt("abc"); // vaut NaN (aucun chiffre au début)
parseInt("2500 FCFA"); // vaut 2500 (lecture stoppée au premier espace)
parseFloat("3.14abc"); // vaut 3,14 (lecture stoppée au premier caractère invalide)
parseInt(""); // vaut NaN (chaîne vide)
```

Remarque bien le premier exemple : `parseInt("12.7")` donne 12 et non 13. Il n'y a pas d'arrondi, seulement une suppression de la partie décimale.

ATTENTION Erreur fréquente : additionner deux résultats de `prompt()`

Le bug le plus classique du débutant est d'écrire directement :

```
let total = prompt("Prix 1 :") + prompt("Prix 2 :");
```

Avec les saisies 2000 et 500, `total` contient la chaîne "2000500" et non le nombre 2500, car `+` concatène deux chaînes. Toute saisie numérique via `prompt()` doit passer par `parseInt()` ou `parseFloat()` avant tout calcul.

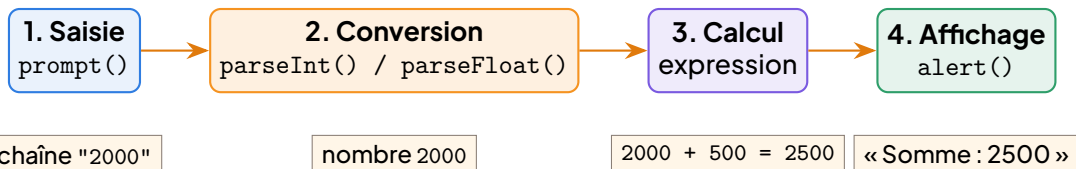
MÉTHODE Lire correctement un nombre saisi par l'utilisateur

Pour lire un nombre et pouvoir calculer avec, applique systématiquement cette démarche en trois étapes :

1. Saisir : récupère la réponse de `prompt()` dans une variable (c'est une chaîne) ;
2. Convertir : applique `parseInt()` (entier) ou `parseFloat()` (réel) à cette chaîne et stocke le résultat ;
3. Calculer et afficher : utilise le nombre obtenu dans tes expressions, puis affiche avec `alert()` ou `document.write()`.

On peut condenser les étapes 1 et 2 en une seule ligne :

```
let age = parseInt(prompt("Quel est ton âge ?"));
```



Chaîne de traitement d'une saisie numérique : la conversion est l'étape indispensable entre `prompt()` (qui produit une chaîne) et le calcul (qui exige des nombres).

LESAVAIS-TU ? NaN est... un nombre !

Voici une curiosité qui surprend tous les débutants : en JavaScript, la valeur `NaN` (Not a Number) est elle-même de type `number` ! L'instruction `typeof NaN` retourne `"number"`. Autre bizarrerie : `NaN` n'est égal à rien, pas même à lui-même (`NaN === NaN` vaut `false`). Pour tester si une conversion a échoué, on utilise donc la fonction prédéfinie `isNaN(valeur)`, qui retourne `true` si la valeur n'est pas un nombre exploitable. C'est un outil précieux pour le contrôle de saisie que tu étudieras en section 6.

3.5 Exemple complet et exercice résolu

EXERCICE RÉSOLU 3 Somme de deux nombres saisis

Énoncé. Le gérant d'un cybercafé de Bafoussam veut une petite page qui demande le prix de deux services (impression, connexion), calcule le total à payer et l'affiche. Écris le script JavaScript complet.

SOLUTION

Solution détaillée. On applique la méthode en trois étapes : saisie, conversion, puis calcul et affichage.

```
// Étape 1 et 2 : saisie + conversion immédiate
let prixImpression = parseInt(prompt("Prix de l'impression (FCFA) :", "0"));
let prixConnexion = parseInt(prompt("Prix de la connexion (FCFA) :", "0"));

// Étape 3 : calcul
let total = prixImpression + prixConnexion;
```

```
// Étape 4 : affichage
alert("Total à payer : " + total + " FCFA");
document.write("Impression : " + prixImpression + " FCFA<br>");
document.write("Connexion : " + prixConnexion + " FCFA<br>");
document.write("<strong>Total : " + total + " FCFA</strong>");
console.log("Total calculé :", total); // trace pour le développeur
```

Trace d'exécution : si l'utilisateur saisit 2000 puis 500, `parseInt` produit les nombres 2000 et 500 ; l'addition donne 2500 ; la boîte de dialogue affiche « Total à payer : 2500 FCFA ». Sans `parseInt()`, on aurait obtenu « 2000500 FCFA ».

EXPÉRIENCE TP guidé : ta première page interactive

Objectif : réaliser une page qui demande l'âge de l'utilisateur et affiche son année de naissance approximative.

Matériel / logiciels : un éditeur de texte (Notepad++, VS Code) et un navigateur (Firefox, Chrome).

Manipulation :

1. Crée un fichier `age.html` et saisis le code suivant :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Calcul de l'année de naissance</title>
</head>
<body>
  <h1>Bienvenue !</h1>
  <script>
    let age = parseInt(prompt("Quel est ton âge ?", "16"));
    let annee = 2025 - age;
    alert("Tu es né(e) vers " + annee + ".");
    document.write("<p>Ton année de naissance est environ <strong>"
      + annee + "</strong>.</p>");
    console.log("Âge saisi :", age, "| Année calculée :", annee);
  </script>
</body>
</html>
```

2. Ouvre le fichier dans le navigateur, saisis ton âge et observe la boîte de dialogue puis la page.
3. Ouvre la console (F12) et vérifie la trace affichée par `console.log()`.
4. Retire volontairement `parseInt()` autour du `prompt()`, recharge et saisis 16 : observe que `2025 - "16"` fonctionne encore (la soustraction force la conversion), mais que `age + 1` donnerait "161". Remets ensuite `parseInt()`.

Observations et interprétation : la page réagit à la saisie de l'utilisateur — c'est déjà un site dynamique côté client. Le TP montre que la conversion explicite avec `parseInt()` rend le code fiable et lisible, sans dépendre des conversions implicites parfois surprenantes de JavaScript.

EXERCICE 3 À toi de jouer

Écris un script qui demande à l'utilisateur le prix unitaire d'un cahier et la quantité achetée (deux `prompt()`), calcule le montant total, puis affiche dans une boîte de dialogue : « Vous devez payer X FCFA ». Prévois le cas où le prix pourrait comporter des décimales : quelle fonction de conversion choisis-tu alors, et pourquoi ?

CORRIGÉ Exercice 1

```
let prixUnitaire = parseFloat(prompt("Prix unitaire du cahier (FCFA) :", "0"));
let quantite = parseInt(prompt("Quantité achetée :", "1"));
let montant = prixUnitaire * quantite;
alert("Vous devez payer " + montant + " FCFA");
```

On choisit `parseFloat()` pour le prix car celui-ci peut comporter une partie décimale (par exemple 350,50 FCFA) : `parseInt()` tronquerait les centimes. En revanche, `parseInt()` suffit pour la quantité, qui est nécessairement un nombre entier de cahiers.

Tu maîtrises désormais le trio fondamental de tout programme interactif : **saisir**, **convertir**, **afficher**. La section suivante te montrera comment faire prendre des décisions à ton script avec les structures alternatives (`if`, `if...else`) et comment répéter des traitements avec les boucles (`for`, `while`, `do...while`) — de quoi rendre tes pages vraiment intelligentes.

4

STRUCTURES ALTERNATIVES ET BOUCLES

Dans la section précédente, nous avons appris à réaliser des **instructions simples** : affectation, saisie avec `prompt()`, affichage avec `alert()` ou `document.write()`, et conversions avec `parseInt()` et `parseFloat()`. Mais un programme qui ne fait qu'exécuter des instructions les unes après les autres, dans l'ordre, reste très limité. Imagine le site de notes de ton lycée : il doit afficher « Admis » si la moyenne est supérieure ou égale à 10, et « Ajourné » sinon. Ou encore un cybercafé de Bonabéri qui doit calculer le montant dû par 50 clients : on ne va pas écrire 50 fois le même calcul ! Il faut donc des instructions capables de **choisir** (les structures alternatives) et de **répéter** (les boucles). C'est l'objet de cette section.

4.1 La structure alternative simple : `if`

DÉFINITION Structure alternative simple

La structure alternative simple **`if`** permet d'exécuter un bloc d'instructions uniquement si une condition est vraie. Si la condition est fausse, le bloc est simplement ignoré et le programme continue après l'accolade fermante.

PROPRIÉTÉ Syntaxe du `if` simple

```
if (condition){  
    // instructions exécutées si condition est vraie  
}
```

La `condition` est une expression booléenne (utilisant les opérateurs de comparaison `==`, `!=`, `<`, `<=`, `>`, `>=` et les opérateurs logiques `&&`, `||`, `!`). Les parenthèses autour de la condition sont obligatoires. Si le bloc ne contient qu'une seule instruction, les accolades peuvent être omises, mais il est fortement conseillé de les mettre toujours pour la lisibilité et la prévention d'erreurs.

EXEMPLE Majorité d'un client

```
let age = parseInt(prompt("Quel est votre âge ?"));  
if (age >= 18){  
    alert("Vous êtes majeur.");  
}  
alert("Merci de votre visite !");
```

Si l'utilisateur saisit 20, les deux messages s'affichent. S'il saisit 15, seul le second message apparaît : le bloc du `if` est sauté.

ATTENTION Valeur `NaN` et comparaisons

Si l'utilisateur annule la boîte de dialogue ou saisit du texte, `parseInt` renvoie `NaN` (Not a Number). Toute comparaison avec `NaN` (`NaN >= 18`, `NaN == NaN`) renvoie `false`. Le bloc `if` ne s'exécute pas, ce qui peut masquer une erreur de saisie. En pratique, on teste souvent `isNaN(age)` avant de comparer.

4.2 La structure alternative complète : `if...else`

Très souvent, on ne veut pas seulement faire quelque chose « si c'est vrai », mais aussi faire autre chose « si c'est faux ». C'est le cas du couple Admis/Ajourné évoqué plus haut.

DÉFINITION Structure alternative complète

La structure **`if...else`** exécute un premier bloc si la condition est vraie, et un second bloc (exclusif du premier) si elle est fausse. Un seul des deux blocs est toujours exécuté.

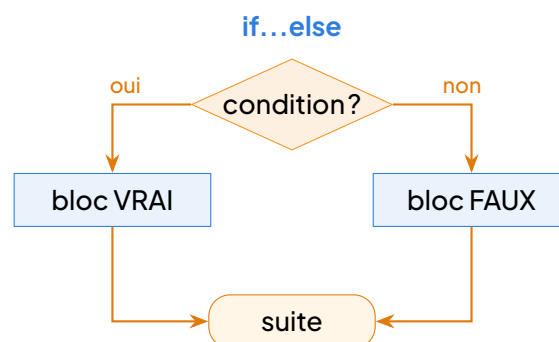
PROPRIÉTÉ Syntaxe du `if...else`

```
if (condition){  
    // instructions si condition VRAIE  
} else {  
    // instructions si condition FAUSSE  
}
```

Le mot-clé `else` ne porte jamais de condition : il signifie « dans tous les autres cas ».

EXEMPLE Décision du jury

```
let moyenne = parseFloat(prompt("Entrez la moyenne :"));  
if (moyenne >= 10){  
    alert("Admis(e) ! Félicitations.");  
} else {  
    alert("Ajourne(e). Courage pour la prochaine fois.");  
}
```



Algorithme de la structure alternative complète : le losange représente le test, et un seul des deux chemins est suivi.

4.3 Tests imbriqués et enchaînement `else if`

Que faire quand il y a plus de deux cas ? Par exemple, la mention au Probatoire : Ajourné, Passable, Assez Bien, Bien, Très Bien. On peut imbriquer des `if` dans des `else`, mais l'enchaînement `else if` est plus lisible : les conditions sont testées **dans l'ordre**, et dès qu'une condition est vraie, son bloc est exécuté et tout le reste est ignoré.

EXEMPLE Mentions au Probatoire

```
let moy = parseFloat(prompt("Moyenne generale :"));
if (moy < 10) {
  alert("Ajourne");
} else if (moy < 12) {
  alert("Mention Passable");
} else if (moy < 14) {
  alert("Mention Assez Bien");
} else if (moy < 16) {
  alert("Mention Bien");
} else {
  alert("Mention Tres Bien");
}
```

L'ordre des tests est crucial : comme on teste d'abord `moy < 10`, dans le premier `else if` on sait déjà que `moy >= 10`, il suffit donc de vérifier `moy < 12`.

ATTENTION Le point-virgule assassin

Ne place jamais de point-virgule juste après la parenthèse d'un `if` ou d'un `while` :

```
if (moy >= 10); // ERREUR : le ; termine le if !
{
  alert("Admis"); // s'exécute TOUJOURS, meme si moy < 10
}
```

Le point-virgule est interprété comme une instruction vide : le `if` ne contrôle plus rien, et le bloc entre accolades s'exécute dans tous les cas. C'est l'un des bugs les plus fréquents et les plus difficiles à repérer, car le code ne produit aucun message d'erreur syntaxique.

4.4 La boucle `for` : un nombre de tours connu

Pour afficher la table de multiplication de 7, on pourrait écrire dix lignes presque identiques. La boucle **for** permet d'écrire le calcul une seule fois et de le répéter un nombre de fois déterminé à l'avance.

PROPRIÉTÉ Syntaxe de la boucle `for`

```
for ( initialisation ; condition ; increment ){  
    // instructions repetees  
}
```

Le déroulement est précisément le suivant :

1. l'initialisation est exécutée une seule fois, au départ (ex. `let i = 1`);
2. la condition est testée : si elle est fausse, la boucle s'arrête immédiatement ;
3. si la condition est vraie, le corps de la boucle s'exécute ;
4. l'incrément est exécuté (ex. `i++`), puis on retourne à l'étape 2.

L'utilisation de `let` pour le compteur (au lieu de `var`) limite sa portée au bloc de la boucle, ce qui évite des effets de bord indésirables.

EXEMPLE Table de multiplication de 7

```
for (let i = 1; i <= 10; i++){  
    document.write("7 x " + i + " = " + (7 * i) + "<br>");  
}
```

Ce script affiche les dix lignes $7 \times 1 = 7$ jusqu'à $7 \times 10 = 70$. La variable `i` est appelée **compteur** de la boucle ; `i++` est un raccourci pour `i = i + 1`.

4.5 La boucle `while` : répéter tant qu'une condition est vraie

Parfois, on ne sait pas à l'avance combien de tours seront nécessaires : on veut simplement continuer tant qu'une condition reste vraie. Par exemple, redemander un mot de passe tant que la saisie est incorrecte.

PROPRIÉTÉ Syntaxe de la boucle `while`

```
while (condition) {  
    // instructions repetees tant que condition est vraie  
}
```

La condition est testée avant chaque tour. Si elle est fausse dès le départ, le corps n'est exécuté aucune fois. Il est indispensable que le corps contienne une instruction qui fasse évoluer la condition, sinon la boucle ne s'arrêtera jamais.

EXEMPLE Saisie contrôlée d'un mot de passe

```
let mdp = prompt("Entrez le mot de passe :");
while (mdp !== "ti2024"){
  mdp = prompt("Incorrect. Reessayez :");
}
alert("Accès autorisé !");
```

Ici, le nombre de tentatives est inconnu à l'avance : c'est exactement le domaine de la boucle `while`. Remarque la nouvelle affectation de `mdp` dans le corps : c'est elle qui permet à la condition d'évoluer.

4.6 La boucle `do...while` : exécuter au moins une fois

La boucle **`do...while`** est une variante de `while` dans laquelle le test est effectué **après** le corps. Conséquence fondamentale : le corps s'exécute **au moins une fois**, même si la condition est fausse dès le départ.

PROPRIÉTÉ Syntaxe de la boucle `do...while`

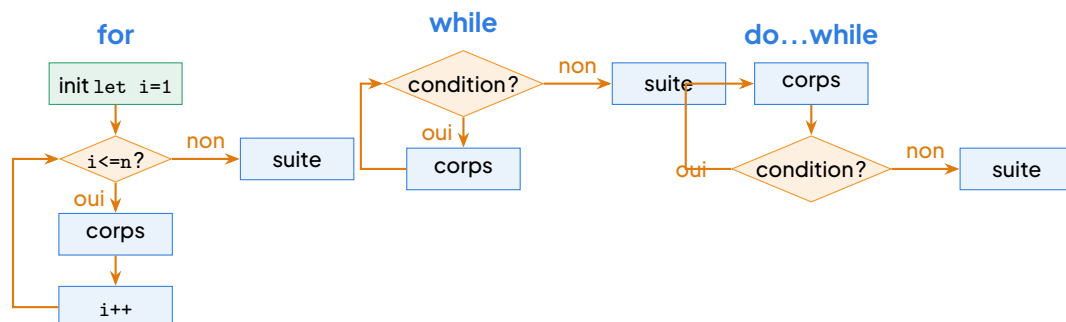
```
do {
  // instructions exécutées au moins une fois
} while (condition);
```

Attention au point-virgule obligatoire après `while (condition)` : c'est la seule boucle qui se termine ainsi.

EXEMPLE Saisie d'une note valide

```
let note;
do {
  note = parseFloat(prompt("Note entre 0 et 20 :"));
} while (note < 0 || note > 20 || isNaN(note));
alert("Note enregistrée : " + note);
```

On doit demander la note au moins une fois avant de pouvoir la tester : le `do...while` est le choix naturel. L'ajout de `isNaN(note)` protège contre une saisie non numérique (annulation ou texte).



Algorithme comparés des trois boucles. Dans `for` et `while`, le test précède le corps ; dans `do...while`, le corps s'exécute d'abord, puis le test décide de la poursuite.

4.7 Choisir sa boucle et éviter la boucle infinie

MÉTHODE Quelle boucle choisir ?

1. Le nombre de tours est connu à l'avance (afficher 10 lignes, traiter 30 élèves, somme des n premiers entiers) : utilise la boucle **for**, qui regroupe initialisation, test et incrément sur une seule ligne.
2. L'arrêt dépend d'une condition et le corps peut ne jamais s'exécuter (tant que le mot de passe est faux, tant que le stock n'est pas épuisé) : utilise la boucle **while**.
3. Le corps doit s'exécuter au moins une fois avant tout test (saisir une valeur puis la valider) : utilise la boucle **do...while**.

En cas de doute entre `for` et `while`, pose-toi la question : « puis-je compter les tours avant de commencer ? ». Si oui : `for`. Si non : `while`.

ATTENTION La boucle infinie

Une boucle `while` (ou `for`) dont la condition n'est jamais modifiée dans le corps est une **boucle infinie** : elle tourne sans fin et bloque l'onglet du navigateur, qui finit par proposer de « tuer » la page.

```

let i = 1;
while (i <= 5) {
    document.write(i + "<br>");
    // OUBLI de i++ : i reste à 1, boucle infinie !
}

```

Avant de tester ton script, vérifie toujours que chaque boucle contient une instruction qui rapproche la condition de la sortie (incrément, nouvelle saisie, modification d'un

drapeau booléen, etc.).

4.8 Trace d'exécution : la somme des n premiers entiers

Pour être sûr qu'une boucle est correcte, les informaticiens réalisent une **trace d'exécution** : ils déroulent le programme « à la main », en notant dans un tableau la valeur de chaque variable à chaque tour. C'est une compétence exigée au Probatoire.

PROPRIÉTÉ Somme des n premiers entiers

La boucle suivante calcule la somme $1 + 2 + \dots + n$:

```
const n = 5;
let s = 0;
for (let i = 1; i <= n; i++) {
  s = s + i;
}
alert("La somme vaut " + s);
```

Vérifions par déroulement pour $n = 5$ que l'on obtient bien $s = 15$ (voir la trace ci-dessous).

Ce résultat est conforme à la formule mathématique $\frac{n(n+1)}{2} = \frac{5 \times 6}{2} = 15$.

Tour	Test $i \leq 5$	i (après $s = s + i$)	s
départ	—	—	0
1	vrai	1	1
2	vrai	2	3
3	vrai	3	6
4	vrai	4	10
5	vrai	5	15
sortie	faux ($i = 6$)	—	15 inchangé

Trace d'exécution de la boucle `for` pour $n = 5$: à chaque tour, s accumule la valeur de i . Au 6^e test, i vaut 6, la condition devient fausse et la boucle s'arrête avec $s = 15$.

EXERCICE RÉSOLU 4 Parité des nombres de 1 à 10

Énoncé. Écris un script JavaScript qui affiche, pour chaque entier de 1 à 10, s'il est pair ou impair. On rappelle que $a \% b$ donne le reste de la division de a par b .

SOLUTION

Solution. Le nombre de tours est connu (10) : on choisit la boucle `for`. À l'intérieur, une structure alternative teste la parité grâce au reste de la division par 2.

```
for (let i = 1; i <= 10; i++){  
  if (i % 2 == 0){  
    document.write(i + " est pair<br>");  
  } else {  
    document.write(i + " est impair<br>");  
  }  
}
```

Trace partielle : pour $i = 1$, $1 \% 2$ vaut 1, donc « 1 est impair » ; pour $i = 2$, $2 \% 2$ vaut 0, donc « 2 est pair » ; et ainsi de suite jusqu'à $i = 10$. Ce script illustre l'imbrication d'une structure alternative dans une boucle, une combinaison très fréquente dans les programmes réels.

EXERCICE 4 Somme des pairs

Écris un script qui calcule et affiche la somme des nombres pairs compris entre 1 et 100, puis vérifie ton résultat par la formule $2 + 4 + \dots + 100 = 2 \times (1 + 2 + \dots + 50) = 50 \times 51 = 2550$.

EXERCICE 5 Devinette

Écris un script qui demande à l'utilisateur de deviner un nombre secret fixé (par exemple 7) en utilisant une boucle `do...while`, et qui affiche « Bravo ! » lorsque le nombre est trouvé. Pourquoi le `do...while` est-il ici préférable au `while` ?

À RETENIR L'essentiel de la section

- `if` exécute un bloc seulement si la condition est vraie ; `if...else` choisit entre deux blocs ; `else if` enchaîne plusieurs cas testés dans l'ordre.
- `for (init ; condition ; incrément)` s'emploie quand le nombre de tours est connu ; `while` répète tant qu'une condition est vraie (corps éventuellement jamais exécuté) ; `do...while` exécute le corps au moins une fois.
- Jamais de point-virgule juste après `if (...)` ou `while (...)` ; toujours une instruction

qui fait évoluer la condition dans le corps d'une boucle, sous peine de boucle infinie.

- La trace d'exécution (tableau des valeurs des variables tour par tour) est l'outil de vérification indispensable d'une boucle.
- Privilégier `let` et `const` (portée de bloc) plutôt que `var` (portée de fonction) pour déclarer les variables et constantes modernes.

5 TABLEAUX, OBJETS ET FONCTIONS

Dans la section précédente, tu as appris à répéter des instructions avec les boucles `for`, `while` et `do...while`. Une question se pose alors naturellement : comment stocker les **notes des 25 élèves** d'une classe de Première TI ? Créer 25 variables `note1`, `note2`, ..., `note25` serait absurde : impossible de les parcourir avec une boucle, puisque chaque nom est différent. Il nous faut une structure capable de regrouper plusieurs valeurs sous un seul nom, repérées par un numéro. Cette structure existe : c'est le **tableau**. Et pour éviter de recopier dix fois le même calcul dans une page, JavaScript nous offre un second outil : la **fonction**, un bloc d'instructions réutilisable à volonté. Cette section te donne ces deux piliers de la programmation, plus une introduction à la création d'objets avec l'opérateur `new`.

5.1 Les tableaux en JavaScript

Le problème à résoudre.

Le proviseur du lycée veut afficher sur le site de l'établissement la moyenne des notes de TI de la classe. Avec des variables ordinaires, il faudrait écrire `somme = n1 + n2 + n3 + ...` jusqu'à la dernière note : un cauchemar dès que la classe compte 40 élèves. Le tableau résout ce problème en donnant un nom unique à l'ensemble et un indice numérique à chaque valeur : la boucle `for` peut alors tout parcourir automatiquement.

DÉFINITION Tableau

Un **tableau** est une structure de données qui regroupe plusieurs valeurs sous un même nom de variable. Chaque valeur est rangée dans une **case** repérée par un **indice** (un numéro de position). En JavaScript, le premier élément porte toujours l'indice 0 : un tableau de n éléments a des indices allant de 0 à $n - 1$.

PROPRIÉTÉ Déclaration et initialisation d'un tableau

Deux syntaxes équivalentes permettent de créer un tableau en JavaScript :

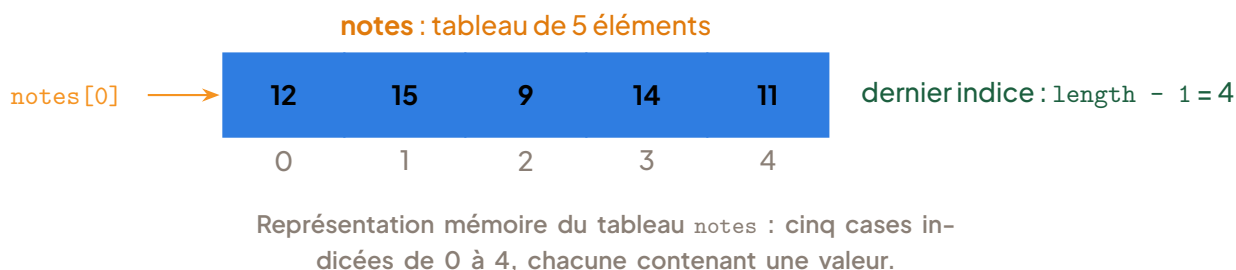
- avec des **crochets** (syntaxe la plus courante, dite littérale) : `var notes = [12, 15, 9];`

- avec l'opérateur `new` et le constructeur `Array` : `var notes = new Array(12, 15, 9);`

On accède ensuite à un élément en écrivant le nom du tableau suivi de l'indice entre crochets : `notes[0]` vaut 12, `notes[1]` vaut 15, `notes[2]` vaut 9. On peut aussi modifier une case : `notes[1] = 17;` remplace 15 par 17.

ATTENTION Piège du constructeur `Array` avec un seul argument numérique

`new Array(5)` ne crée pas un tableau contenant le nombre 5, mais un tableau vide de longueur 5 (5 cases `undefined`). Pour créer un tableau à un seul élément numérique, utilise obligatoirement la syntaxe à crochets : `[5]`.



PROPRIÉTÉ La propriété `length`

Tout tableau JavaScript possède la propriété `length`, qui donne le nombre d'éléments du tableau. Si `notes` contient 5 notes, alors `notes.length` vaut 5. Comme les indices commencent à 0, le dernier indice valide est `notes.length - 1`.

ATTENTION L'erreur classique : dépasser le dernier indice

L'erreur la plus fréquente des débutants est d'écrire `notes[notes.length]`. Si le tableau contient 5 éléments, les indices valides sont 0, 1, 2, 3, 4 : la case d'indice 5 n'existe pas. JavaScript ne signale pas d'erreur, il retourne silencieusement `undefined`, ce qui fausse tous les calculs (une somme devient `NaN`). Retiens : le dernier indice valide est toujours `length - 1`. Dans une boucle, écris `i < notes.length` et jamais `i <= notes.length`.

Parcours d'un tableau avec une boucle `for`.

C'est le mariage parfait entre cette section et la précédente : la boucle `for` fait varier l'indice `i` de 0 à `length - 1`, et chaque tour traite la case `notes[i]`.

EXEMPLE Afficher toutes les notes d'un tableau

```
var notes = [12, 15, 9, 14, 11];  
  
// i prend successivement les valeurs 0, 1, 2, 3, 4  
for (var i = 0; i < notes.length; i++) {  
    document.write("Note n° " + (i + 1) + " : " + notes[i] + "<br>");  
}
```

Ce script affiche les cinq notes, numérotées de 1 à 5 pour l'utilisateur (on écrit `i + 1` car l'être humain compte à partir de 1, la machine à partir de 0).

Recherche d'un élément dans un tableau.

Autre besoin concret : la secrétaire du lycée veut savoir si un élève a obtenu la note 20 quelque part dans le tableau, et à quelle position. Il faut parcourir le tableau en comparant chaque case à la valeur cherchée, tout en mémorisant si on l'a rencontrée.

MÉTHODE Rechercher un élément `x` dans un tableau (première occurrence)

1. Initialiser un indicateur booléen `trouve` à `false` et une variable `position` à `-1` (valeur conventionnelle signifiant « pas trouvé »).
2. Parcourir le tableau avec une boucle `for` (indice `i` de 0 à `length - 1`).
3. À chaque tour, comparer la case courante à `x` : si `t[i] == x`, alors mettre `trouve` à `true`, mémoriser `position = i` et quitter la boucle avec `break`.
4. Après la boucle, tester `trouve` : s'il vaut `true`, afficher `position`; sinon annoncer que l'élément est absent.

EXEMPLE Recherche de la note 20 dans le tableau

```
var notes = [12, 15, 20, 9, 14];  
var x = 20;           // valeur recherchée  
var trouve = false;   // indicateur  
var position = -1;     // -1 signifie "pas trouvé"  
  
for (var i = 0; i < notes.length; i++) {  
    if (notes[i] == x) {  
        trouve = true;  
        position = i; // on mémorise l'indice  
        break;       // on stoppe la boucle : première occurrence trouvée  
    }  
}
```

```
if (trouve){  
    alert("La note " + x + " existe à l'indice " + position);  
} else {  
    alert("La note " + x + " n'existe pas dans le tableau.");  
}
```

Ici, la recherche s'arrête sur l'indice 2 : le script affiche « La note 20 existe à l'indice 2 ».

5.2 Création d'objets avec l'opérateur new

JavaScript fournit des **objets** prédéfinis, c'est-à-dire des structures riches dotées de propriétés (des données) et de méthodes (des actions). On les crée avec l'opérateur **new** suivi du nom du constructeur :

```
var d = new Date();      // objet Date : date et heure actuelles  
var t = new Array();     // tableau vide (on ajoutera les cases ensuite)  
var s = new String("TI"); // objet chaîne de caractères
```

L'objet `Date` est très utile pour un site dynamique : il capture la date et l'heure du moment où la page s'exécute, et ses méthodes permettent d'en extraire les composantes.

EXEMPLE Afficher la date du jour sur la page d'accueil du lycée

```
var d = new Date();      // création de l'objet Date  
var jour = d.getDate();  // numéro du jour du mois (1 à 31)  
var mois = d.getMonth() + 1; // mois : attention, 0 = janvier !  
var annee = d.getFullYear(); // année sur 4 chiffres  
  
document.write("Nous sommes le " + jour + "/" + mois + "/" + annee);
```

ATTENTION Le piège de getMonth()

La méthode `getMonth()` retourne les mois numérotés de 0 à 11 (0 pour janvier, 11 pour décembre), comme les indices d'un tableau. Oublier le + 1 affiche un mois de retard : en mars, ta page annoncerait « mois 2 ». Pense toujours à ajouter 1.

5.3 Les fonctions

Le problème à résoudre.

Sur le site du lycée, on calcule la moyenne des notes de la classe de Première, puis celle de Terminale, puis celle de Seconde. Faut-il recopier trois fois la même boucle de somme ? Non : on écrit le calcul une seule fois dans une **fonction**, puis on l'appelle autant de fois que nécessaire

avec des données différentes. La fonction est une machine : on lui donne des ingrédients (les **paramètres**), elle effectue un traitement, et elle peut rendre un résultat (la **valeur de retour**).



Une fonction vue comme une machine : les paramètres entrent, le traitement s'exécute, la valeur de retour sort.

PROPRIÉTÉ Déclaration et appel d'une fonction

La syntaxe générale de déclaration est :

```
function nomFonction(parametre1, parametre2) {  
    // instructions du traitement  
    return expression; // seulement si la fonction retourne une valeur  
}
```

- Le mot-clé `function` est suivi du nom de la fonction, puis des paramètres entre parenthèses (séparés par des virgules ; les parenthèses restent obligatoires même vides), puis du corps entre accolades.
- Fonction sans retour (appelée aussi procédure) : elle exécute des actions (affichage, modification de la page) mais ne rend rien ; elle ne contient pas de `return` (ou un `return` seul sans expression).
- Fonction avec retour : l'instruction `return expression;` calcule l'expression, renvoie sa valeur à l'appelant et termine immédiatement la fonction.
- Pour appeler la fonction, on écrit son nom suivi des valeurs concrètes (les arguments) :
`var m = moyenne(notes);` récupère la valeur retournée dans la variable `m`.

MÉTHODE Écrire une fonction en trois questions

1. Quelles sont les entrées ? Identifie les données dont le traitement a besoin : ce seront les paramètres, placés entre parenthèses.
2. Quel est le traitement ? Écris les instructions qui transforment les entrées en résultat (boucles, calculs, tests).
3. Faut-il rendre un résultat ? Si l'appelant doit récupérer une valeur (une moyenne, un total, un vrai/faux), termine par `return` suivi de cette valeur. Si la fonction ne fait qu'afficher ou modifier la page, aucun `return` n'est nécessaire.

EXEMPLE Fonction sans retour : afficher un message de bienvenue

```
// Procédure : elle affiche, mais ne retourne rien
function bienvenue(nom){
    alert("Bienvenue " + nom + " sur le site du lycée !");
}

bienvenue("Amina"); // appel : affiche "Bienvenue Amina ..."
bienvenue("Biya"); // on réutilise la même fonction
```

EXERCICE RÉSOLU 5 Calculer la moyenne d'un tableau de notes

Énoncé. Écrire une fonction `moyenne(t)` qui reçoit un tableau de notes, parcourt toutes ses cases, calcule la somme des éléments et retourne la moyenne. Vérifier que `moyenne([10, 12, 14])` retourne 12.

SOLUTION

Solution détaillée. On applique la méthode en trois questions : l'entrée est le tableau `t` (un seul paramètre) ; le traitement est une boucle de somme ; le résultat doit être rendu, donc on utilise `return`. On prévoit le cas d'un tableau vide pour éviter une division par zéro.

```
function moyenne(t){
    if (t.length === 0) return 0; // garde-fou : tableau vide
    var somme = 0; // accumulateur initialisé à 0
    for (var i = 0; i < t.length; i++){ // parcours des indices 0 à length-1
        somme = somme + t[i]; // on ajoute chaque case
    }
    return somme / t.length; // valeur retournée
}

// Appel de la fonction et récupération du résultat
var m = moyenne([10, 12, 14]);
alert("La moyenne est : " + m); // affiche 12
```

Vérification pas à pas : `somme` vaut successivement 0, puis $0 + 10 = 10$, puis $10 + 12 = 22$, puis $22 + 14 = 36$. Le tableau contient 3 éléments, donc la fonction retourne $36/3 = 12$. L'appel stocke 12 dans `m`, que l'on affiche. Remarque que la même fonction fonctionne pour n'importe quel tableau : `moyenne([8, 9])` retourne 8,5.

Portée des variables.

Une variable déclarée avec `var` à l'intérieur d'une fonction est dite **locale** : elle n'existe que pendant l'exécution de cette fonction et est invisible à l'extérieur. Dans l'exemple précédent, `somme` et `i` sont locales à `moyenne` : après l'appel, elles ont disparu de la mémoire. C'est une protection précieuse : deux fonctions peuvent utiliser chacune une variable `i` sans se gêner. À l'inverse, une variable déclarée hors de toute fonction est **globale** : elle est accessible partout dans le

script.

ATTENTION Confondre paramètre et variable globale

Un piège fréquent consiste à utiliser dans une fonction une variable globale au lieu d'un paramètre : la fonction « marche » tant que la globale porte le bon nom, puis casse dès qu'on la réutilise ailleurs. Règle d'or : tout ce dont une fonction a besoin doit entrer par ses paramètres, et tout ce qu'elle produit doit sortir par `return`. Ainsi, `moyenne(t)` fonctionne avec n'importe quel tableau, pas seulement avec celui qui s'appelle `notes`.

EXPÉRIENCE TP guidé : statistiques des notes d'une classe

Objectif : écrire une page qui affiche la moyenne, la note maximale et le nombre d'admis (note ≥ 10) d'une classe.

Logiciels : un éditeur de texte (Notepad++) et un navigateur (Firefox ou Chrome).

Manipulation :

1. Crée un fichier `stats.html` et saisis le code suivant :

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Statistiques de la classe</title>
</script>
  function moyenne(t) {
    if (t.length === 0) return 0;
    var somme = 0;
    for (var i = 0; i < t.length; i++) {
      somme = somme + t[i];
    }
    return somme / t.length;
  }

  function maximum(t) {
    if (t.length === 0) return undefined;
    var max = t[0]; // on suppose la première case la plus grande
    for (var i = 1; i < t.length; i++) {
      if (t[i] > max) {
        max = t[i];
      }
    }
    return max;
  }

  function nombreAdmis(t) {
    var compteur = 0;
    for (var i = 0; i < t.length; i++) {
      if (t[i] >= 10) {
        compteur = compteur + 1;
      }
    }
  }
}
```

```
        }  
    }  
    return compteur;  
}  
</script>  
</head>  
<body>  
  <h1>Résultats de la classe de Première TI</h1>  
  <script>  
    var notes = [12, 8, 15, 9, 14, 11, 7, 16];  
    document.write("Moyenne : " + moyenne(notes) + "<br>");  
    document.write("Note maximale : " + maximum(notes) + "<br>");  
    document.write("Admis : " + nombreAdmis(notes) + " / " + notes.length);  
  </script>  
</body>  
</html>
```

2. Enregistre, puis ouvre le fichier dans le navigateur.

3. Modifie le contenu du tableau `notes`, enregistre et recharge la page (touche F5).

Observations : la page affiche la moyenne 11,5, le maximum 16 et 6 admis sur 8. Après modification du tableau, les trois résultats se mettent à jour sans toucher aux fonctions.

Interprétation : les trois fonctions sont déclarées dans l'en-tête et appelées dans le corps avec le même tableau. Chaque fonction applique un algorithme classique de parcours : accumulation (somme), comparaison mémorisée (maximum) et comptage conditionnel (admis). C'est toute la puissance du couple tableau + fonction : un seul code, des données interchangeables.

À RETENIR L'essentiel de la section

- Un **tableau** regroupe plusieurs valeurs sous un même nom ; chaque case est repérée par un **indice** qui commence à 0. On le crée avec `var t = [12, 15, 9];` ou `new Array(12, 15, 9)` (attention : `new Array(5)` crée un tableau vide de longueur 5).
- `t.length` donne le nombre d'éléments ; le dernier indice valide est `t.length - 1`. On parcourt un tableau avec `for (var i = 0; i < t.length; i++)`.
- L'opérateur `new` crée des objets prédéfinis : `new Date()`, `new Array()`.
- Une **fonction** se déclare avec `function nom(parametres) { ... }` et s'appelle par son nom suivi des arguments. `return` renvoie une valeur et termine la fonction ; sans `return`, c'est une procédure qui agit sans rien rendre.
- Une variable déclarée dans une fonction est **locale** : elle n'existe qu'à l'intérieur de celle-ci.

EXERCICE 6 À toi de jouer

1. Écris une fonction `minimum(t)` qui retourne la plus petite note d'un tableau (on suppose le tableau non vide). Teste-la avec `[12, 8, 15, 9]`.
2. Écris une fonction `compter(t, x)` qui retourne le nombre de fois où la valeur `x` apparaît dans le tableau `t`.
3. Écris une procédure `afficherTableau(t)` qui affiche tous les éléments d'un tableau, un par ligne, précédés de leur indice.
4. Que vaut `notes[notes.length]` si `notes = [5, 7, 9]` ? Explique pourquoi.

CORRIGÉ Exercice n 1

```
function minimum(t) {  
  var min = t[0]; // première case comme candidat  
  for (var i = 1; i < t.length; i++) {  
    if (t[i] < min) {  
      min = t[i];  
    }  
  }  
  return min;  
}
```

Avec `[12, 8, 15, 9]`, `min` vaut 12 puis 8 ; les cases 15 et 9 ne le battent pas. La fonction retourne 8.

CORRIGÉ Exercice n 2

```
function compter(t, x) {  
  var compteur = 0;  
  for (var i = 0; i < t.length; i++) {  
    if (t[i] == x) {  
      compteur = compteur + 1;  
    }  
  }  
  return compteur;  
}
```

C'est l'algorithme de recherche vu en cours, sauf qu'on ne s'arrête pas à la première occurrence : on compte toutes les cases égales à `x`.

CORRIGÉ Exercice n 3

```
function afficherTableau(t) {  
    for (var i = 0; i < t.length; i++) {  
        document.write("Indice " + i + " : " + t[i] + "<br>");  
    }  
}
```

C'est une procédure : elle affiche et ne retourne rien, donc aucun `return`.

CORRIGÉ Exercice n 4

`notes.length` vaut 3, donc `notes[notes.length]` est `notes[3]`. Or les indices valides sont 0, 1 et 2 : la case 3 n'existe pas. JavaScript retourne `undefined`. Le dernier indice valide est toujours `length - 1`, ici `notes[2]` qui vaut 9.

6 ÉVÈNEMENTS ET CONTRÔLE DES FORMULAIRES

Dans la section précédente, nous avons appris à structurer des données complexes avec les tableaux et les objets, et à factoriser nos traitements grâce aux fonctions. Nous possédons désormais la « boîte à outils » algorithmique de JavaScript. Il nous reste à brancher ces outils sur l'interface utilisateur : comment faire en sorte qu'un clic sur un bouton déclenche un calcul, qu'une saisie dans un champ soit vérifiée instantanément, ou que la page s'initialise correctement au chargement ? C'est tout l'objet de la **programmation évènementielle**, cœur de l'interactivité côté client.

6.1 Programmation évènementielle : concepts et insertion

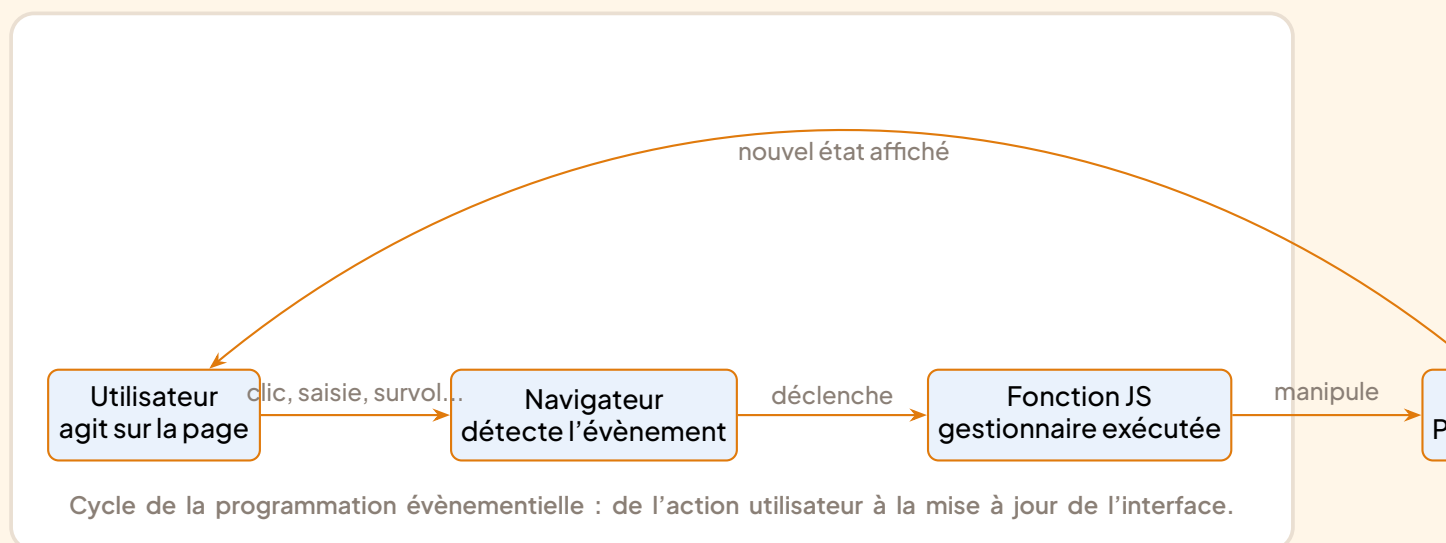
DÉFINITION Évènement et gestionnaire d'évènement

Un **évènement** est une action détectée par le navigateur : un clic de souris, l'appui sur une touche, le chargement d'une image, la modification d'un champ de formulaire, etc. Un **gestionnaire d'évènement** (ou écouteur) est une fonction JavaScript qui sera exécutée automatiquement lorsque l'évènement se produit sur un élément HTML précis.

En JavaScript côté client (niveau Première TI), la méthode historique et pédagogique pour associer un évènement à une fonction consiste à utiliser un **attribut HTML** directement dans la balise de l'élément concerné. La syntaxe générale est :

```
<balise onevent="nomFonction();">
```

où `onevent` est le nom de l'attribut (ex. `onclick`, `onchange`) et `nomFonction` le nom de la fonction JavaScript définie dans un bloc `<script>`.



6.2 Les principaux événements du DOM

Le tableau ci-dessous récapitule les cinq événements incontournables du programme officiel, leurs cibles habituelles et leurs cas d'usage types dans une application web camerounaise (ex. : gestion de stock d'une boutique à Douala, inscription à un concours à l'université de Yaoundé I).

Attribut HTML	Éléments cibles typiques	Utilité pédagogique / Pro
<code>onclick</code>	<code><button></code> , <code><a></code> , <code><div></code> , <code></code> , tout élément visible	Déclencher une action majeure : valider un formulaire, ouvrir une modale, lancer un calcul, supprimer une ligne de tableau.
<code>onmouseover</code>	<code><button></code> , <code><a></code> , <code><tr></code> , <code><div></code>	Effet visuel de survol (tooltip, changement de couleur, affichage d'aide contextuelle) sans clic.
<code>onchange</code>	<code><input></code> , <code><select></code> , <code><textarea></code>	Détecter la fin de modification d'un champ (perte de focus après changement). Idéal pour : recalculer un total, filtrer une liste, vérifier un format.
<code>onfocus</code>	<code><input></code> , <code><select></code> , <code><textarea></code> , <code><button></code>	Réagir à l'entrée dans un champ : afficher une aide, vider un placeholder, mettre en surbrillance la ligne du formulaire.
<code>onload</code>	<code><body></code> , <code></code> , <code><script></code> , <code><iframe></code>	Exécuter du code après le chargement complet de la ressource. Sur <code><body></code> : initialiser l'application, charger des données, lancer une animation.

Note : `onblur` (perte de focus) et `onmouseout` (fin de survol) sont les pairs naturels de `onfocus` et `onmouseover` ; ils sont très utilisés ensemble pour restaurer l'état initial de l'interface.

EXEMPLE Illustration de chaque évènement

Le code suivant montre la syntaxe exacte pour chaque évènement. Ouvrez la console (F12) pour voir les messages.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Démonstration Evènements JS</title>
  <style>
    .zone { padding : 10px ; margin : 5px ; border : 1px solid #ccc ; background : #f9f9f9 ; }
    .survol { background : #ffe0b2 ; transition : background 0.3s ; }
    .focus { border-color : #2196f3 ; box-shadow : 0 0 5px #2196f3 ; }
  </style>
  <script>
    // 1. onClick
    function direBonjour() {
      alert("Bienvenue à la boutique 'Au Bon Prix' !");
      console.log("Évènement : click sur le bouton");
    }

    // 2. onMouseOver / onMouseOut
    function survoler(elem) { elem.classList.add("survol") ; console.log("Survol ON") ; }
    function quitter(elem) { elem.classList.remove("survol") ; console.log("Survol OFF") ; }

    // 3. onChange
    function verifierChoix(select) {
      const val = select.value ;
      console.log("Choix modifié : " + val) ;
      if (val === "autre") {
        alert("Merci de préciser dans le champ suivant.") ;
      }
    }

    // 4. onFocus / onBlur
    function aideChamp(input) {
      input.classList.add("focus") ;
      input.placeholder = "Entrez votre texte ici..." ;
    }
    function quitterChamp(input) {
      input.classList.remove("focus") ;
      input.placeholder = "" ;
    }

    // 5. onLoad (sur body)
    function initialiserPage() {
      console.log("Page entièrement chargée (DOM + images + CSS).") ;
      document.getElementById("annee").textContent = new Date().getFullYear() ;
    }
  </script>

```

```
</script>
</head>
<body onload="initialiserPage()">
  <h1>Test des évènements JavaScript</h1>

  <!-- onClick -->
  <button onclick="direBonjour()">Cliquez-moi (onClick)</button>

  <hr>

  <!-- onMouseOver / onMouseOut -->
  <div class="zone" onmouseover="survoler(this)" onmouseout="quitter(this)">
    Passez la souris ici (onMouseOver / onMouseOut)
  </div>

  <hr>

  <!-- onChange -->
  <label for="categorie">Catégorie (onChange) : </label>
  <select id="categorie" onchange="verifierChoix(this)">
    <option value="">-- Choisir --</option>
    <option value="boisson">Boissons</option>
    <option value="alimentaire">Alimentaire</option>
    <option value="autre">Autre...</option>
  </select>

  <hr>

  <!-- onFocus / onBlur -->
  <label for="commentaire">Commentaire (onFocus/onBlur) : </label><br>
  <textarea id="commentaire" rows="3" cols="40"
    onfocus="aideChamp(this)" onblur="quitterChamp(this)">
  </textarea>

  <hr>
  <p>Année courante (injectée par onLoad) : <span id="annee"></span></p>
</body>
</html>
```

ATTENTION Piège classique : `this` dans les attributs HTML

Dans un attribut événementiel comme `onclick="maFonction(this)"`, le mot-clé `this` représente l'élément HTML lui-même (l'objet DOM). C'est la méthode la plus simple pour passer l'élément à la fonction sans avoir à faire un `getElementById` à l'intérieur. N'oubliez pas les parenthèses après le nom de la fonction dans l'attribut HTML : `onclick="maFonction()"` et non `onclick="maFonction"`.

6.3 Accès aux éléments HTML : `document.getElementById` et propriété `.value`

Pour contrôler une saisie ou modifier le contenu d'une page, le script doit « attraper » l'élément HTML. La méthode standard au programme est `document.getElementById("identifiant")`.

PROPRIÉTÉ Accès à un élément par son id

- **Syntaxe** : `var element = document.getElementById("id_de_l_element");`
- **Condition** : L'élément HTML doit posséder un attribut `id="id_de_l_element"` unique dans la page.
- **Retour** : Un objet `HTMLElement` (ou `null` si introuvable).
- **Propriété clé** : `element.value` renvoie la valeur courante d'un champ de formulaire (`<input>`, `<select>`, `<textarea>`).

ATTENTION Erreurs fréquentes — À lire avant tout TP

1. **Guillemets obligatoires** : `getElementById("champ")` (chaîne de caractères). `getElementById(champ)` cherche une variable JS nommée `champ` → `ReferenceError`.
2. **id vs name** : `getElementById` ne cherche que l'attribut `id`. L'attribut `name` sert à l'envoi des données au serveur (PHP en Terminale), pas à la sélection JS côté client.
3. **.value est toujours une chaîne** : Même pour `<input type="number">`, `element.value` renvoie "12" (string), pas 12 (number). Conversion impérative avec `parseInt()` ou `parseFloat()` avant tout calcul ou comparaison numérique.
4. **Exécution trop tôt** : Si le script dans `<head>` appelle `getElementById` avant que le `<body>` ne soit lu, l'élément n'existe pas encore → retourne `null`. Solution : placer le script à la fin du `<body>` ou utiliser `window.onload / <body onload="...">`.

EXEMPLE Lecture, conversion et calcul

```
function calculerMontant() {  
    // 1. Récupérer les éléments  
    const inputQte = document.getElementById("qte");  
    const inputPU = document.getElementById("pu");  
    const spanTotal = document.getElementById("total");  
  
    // 2. Lire les valeurs (chaînes)  
    const strQte = inputQte.value; // ex : "5"  
    const strPU = inputPU.value; // ex : "1200"
```

```
// 3. CONVERSION IMPÉRATIVE
const qte = parseInt(strQte, 10); // base 10 recommandée
const pu = parseFloat(strPU); // gère les décimales

// 4. Vérification basique (NaN = Not a Number)
if (isNaN(qte) || isNaN(pu) || qte <= 0 || pu < 0) {
    spanTotal.textContent = "Erreur de saisie";
    spanTotal.style.color = "red";
    return; // stoppe la fonction
}

// 5. Calcul et affichage
const montant = qte * pu;
spanTotal.textContent = montant.toLocaleString() + " FCFA"; // formatage FR
spanTotal.style.color = "green";
}
```

6.4 Contrôle de saisie et validation de formulaire

Le contrôle de saisie (validation côté client) vise à donner un retour **immédiat** à l'utilisateur avant d'envoyer les données au serveur. Cela économise de la bande passante et améliore l'expérience utilisateur (UX).

MÉTHODE Contrôler une saisie en 4 étapes

1. Récupérer l'élément avec `document.getElementById("idChamp")`.
2. Lire la valeur via la propriété `.value` (rappel : c'est une `String`).
3. Tester la valeur :
 - Non vide : `val.trim() !== ""`
 - Entier positif strict : `!isNaN(val) && parseInt(val, 10) > 0 && val === String(parseInt(val, 10))`
 - Format complexe (email, téléphone CM) : Expressions régulières (voir Terminale) ou logique if simple (longueur, préfixe "6" ou "2" pour le Cameroun).
4. Réagir :
 - Afficher un message : `alert("Message")` (modal, bloquant) ou injection dans un `<span>` (non bloquant, pro).
 - Bloquer l'envoi : la fonction de validation appelée par `onsubmit` doit retourner `false` si erreur.

PROPRIÉTÉ Validation globale : l'attribut `onsubmit` sur `<form>`

Pour interdire l'envoi du formulaire si les données sont invalides, on place un gestionnaire sur la balise `<form>` :

```
<form action="traitement.php" method="post" onsubmit="return
verifierFormulaire();">
```

- La fonction `verifierFormulaire()` doit retourner `true` (envoi autorisé) ou `false` (envoi annulé).
- Le `return` devant l'appel dans l'attribut est crucial : `onsubmit="return verifierFormulaire()"`.

EXEMPLE Vérification quantité entière positive — Exemple chiffré

```
function verifierQuantite() {
    const champ = document.getElementById("qteCmd");
    const val = champ.value.trim(); // " 12 " -> "12"

    // 1. Non vide
    if (val === "") {
        alert("La quantité est obligatoire.");
        champ.focus(); // replace le curseur
        return false;
    }

    // 2. Conversion et test entier positif
    const qte = parseInt(val, 10);
    if (isNaN(qte) || qte <= 0 || val !== String(qte)) {
        // val !== String(qte) détecte "12.5" ou "12abc" -> parseInt donne 12 mais val != "12"
        alert("Quantité invalide : veuillez entrer un entier strictement positif (ex : 5).");
        champ.select(); // sélectionne le texte pour correction facile
        return false;
    }

    return true; // OK
}
```

6.5 Exemple complet : Formulaire de commande avec calcul automatique et validation

Scénario : Une petite entreprise de vente de matériel informatique à Bafoussam veut une page web simple pour ses commerciaux itinérants. Ils saisissent le client, choisissent un produit (prix fixe), entrent la quantité, le montant se calcule instantanément (`onChange`), et la validation finale (`onSubmit`) vérifie que le client n'est pas vide et que la quantité est un entier > 0 .

≡ BON DE COMMANDE — SARL TECHNO+

onFocus / onBlur ➤ Client : `<input id="client" type="text" onfocus="effacerAide(this)" onblur="verifierC">`

onChange ➤ Produit : `<select id="produit" onchange="calculerTotal()">`

onChange ➤ Quantité : `<input id="qte" type="text" onchange="calculerTotal()">`

Prix unitaire : `<output id="pu">0</output>` FCFA

Montant total : `<output id="total" style="color:popgreen">0</output>` FCFA

onSubmit (form) ➤ `<button type="submit">Valider la commande</button>`

Maquette du formulaire de commande annotée avec les événements JavaScript associés à chaque champ.

EXERCICE RÉSOLU 6 Formulaire de commande complet — Code source commenté

Énoncé : Réaliser la page HTML/JavaScript complète correspondant à la maquette ci-dessus. Le prix unitaire dépend du produit choisi (Souris : 5 000 FCFA, Clavier : 12 000 FCFA, Écran : 85 000 FCFA). La validation finale vérifie : client non vide, quantité entier > 0. En cas de succès, afficher un récapitulatif via `alert()` et laisser le formulaire s'envoyer (simulé ici par `return true`).

Solution détaillée :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Bon de Commande – SARL TECHNO+</title>
  <style>
    body { font-family : Segoe UI, Arial, sans-serif; max-width : 500px; margin : 2rem auto; padding : 1rem; background : #f4f6f9; }
    fieldset { border : 1px solid #ddd; border-radius : 8px; background : white; padding : 1.5rem; box-shadow : 0 2px 5px rgba(0,0,0,0.05); }
    legend { font-weight : bold; color : #2c3e50; padding : 0 0.5rem; }
```

```
.champ { margin-bottom : 1rem; display : flex; flex-direction : column; gap : 0.3rem; }
label { font-weight : 600; font-size : 0.9rem; color : #34495e; }
input, select { padding : 0.6rem; border : 1px solid #bdc3c7; border-radius : 4px; font-size : 1rem;
}
input :focus, select :focus { outline : none; border-color : #3498db; box-shadow : 0 0 0 3px rgba
(52,152,219,0.2); }
input.erreur { border-color : #e74c3c; background : #fdf2f2; }
output#pu, output#total { font-weight : bold; color : #27ae60; font-size : 1.1rem; }
button { width : 100%; padding : 0.8rem; background : #2c3e50; color : white; border : none;
border-radius : 4px; font-size : 1rem; cursor : pointer; transition : background 0.2s; }
button :hover { background : #1a252f; }
.aide { font-size : 0.8rem; color : #7f8c8d; min-height : 1.2em; }
.aide.erreur { color : #e74c3c; }
</style>
<script>
// --- DONNÉES (Modèle simple) ---
const CATALOGUE = {
  "souris" : { nom : "Souris sans fil Logitech", prix : 5000 },
  "clavier" : { nom : "Clavier mécanique AZERTY", prix : 12000 },
  "ecran" : { nom : "Écran 24 pouces IPS", prix : 85000 }
};

// --- FONCTIONS UTILITAIRES ---
function afficherErreur(idChamp, message) {
  const champ = document.getElementById(idChamp);
  const aide = document.getElementById(idChamp + "Aide");
  champ.classList.add("erreur");
  if (aide) { aide.textContent = message; aide.classList.add("erreur"); }
}
function effacerErreur(idChamp) {
  const champ = document.getElementById(idChamp);
  const aide = document.getElementById(idChamp + "Aide");
  champ.classList.remove("erreur");
  if (aide) { aide.textContent = ""; aide.classList.remove("erreur"); }
}

// --- 1. onFocus / onBlur sur Client ---
function effacerAide(input) { effacerErreur(input.id); }
function verifierClient(input) {
  if (input.value.trim() === "") afficherErreur("client", "Le nom du client est obligatoire.");
}

// --- 2. onChange sur Produit : met à jour PU ---
function majPrixUnitaire() {
  const select = document.getElementById("produit");
  const code = select.value;
  const puEl = document.getElementById("pu");
  if (code && CATALOGUE[code]) {
    puEl.textContent = CATALOGUE[code].prix.toLocaleString();
  } else {
    puEl.textContent = "0";
  }
  calculerTotal(); // recalcul si quantité déjà saisie
}
```

```
}

// --- 3. onChange sur Quantité : calcule Total ---
function calculerTotal() {
    const qteStr = document.getElementById("qte").value.trim();
    const puText = document.getElementById("pu").textContent.replace(/\\s/g, ""); // retire
    espaces insécables
    const totalEl = document.getElementById("total");

    const qte = parseInt(qteStr, 10);
    const pu = parseInt(puText, 10);

    if (!isNaN(qte) && qte > 0 && !isNaN(pu) && pu > 0) {
        totalEl.textContent = (qte * pu).toLocaleString();
        totalEl.style.color = "#27ae60";
        effacerErreur("qte");
    } else {
        totalEl.textContent = "0";
        totalEl.style.color = "#e74c3c";
        if (qteStr !== "") afficherErreur("qte", "Entier strictement positif requis.");
    }
}

// --- 4. onSubmit : Validation globale ---
function verifierFormulaire() {
    let valide = true;

    // Client
    const client = document.getElementById("client");
    if (client.value.trim() === "") {
        afficherErreur("client", "Le nom du client est obligatoire.");
        valide = false;
    } else effacerErreur("client");

    // Quantité
    const qteStr = document.getElementById("qte").value.trim();
    const qte = parseInt(qteStr, 10);
    if (qteStr === "" || isNaN(qte) || qte <= 0 || qteStr !== String(qte)) {
        afficherErreur("qte", "Quantité invalide (entier > 0).");
        valide = false;
    } else effacerErreur("qte");

    // Produit choisi ?
    if (document.getElementById("produit").value === "") {
        afficherErreur("produit", "Veuillez choisir un produit.");
        valide = false;
    } else effacerErreur("produit");

    if (!valide) {
        // Scroll vers la première erreur (compatible ES5)
        var premierErreur = document.querySelector(".erreur");
        if (premierErreur) premierErreur.scrollIntoView({ behavior: "smooth", block: "center" });
        return false; // Bloque l'envoi (standard 1ère TI)
    }
}
```

```

    }

    // Succès : récapitulatif
    const prodCode = document.getElementById("produit").value;
    const prod = CATALOGUE[prodCode];
    const recap = "COMMANDE VALIDÉE\n\nClient : " + client.value.trim() +
        "\nProduit : " + prod.nom +
        "\nQuantité : " + qte +
        "\nPrix unitaire : " + prod.prix.toLocaleString() + " FCFA" +
        "\nTOTAL : " + (qte * prod.prix).toLocaleString() + " FCFA";
    alert(recap);
    return true; // Envoi effectif (vers serveur en Terminale)
}

// --- Initialisation au chargement (onLoad) ---
function initialiser () {
    // Remplir le select dynamiquement depuis l'objet CATALOGUE
    const select = document.getElementById("produit");
    for (const code in CATALOGUE) {
        const opt = document.createElement("option");
        opt.value = code;
        opt.textContent = CATALOGUE[code].nom + " (" + CATALOGUE[code].prix.toLocaleString
() + " FCFA)";
        select.appendChild(opt);
    }
    document.getElementById("annee").textContent = new Date().getFullYear();
}
</script>
</head>
<body onload="initialiser()">
    <form action="#" method="post" onsubmit="return verifierFormulaire()">
        <fieldset>
            <legend>≡ Nouveau Bon de Commande</legend>

            <div class="champ">
                <label for="client">Client <span style="color:red">*</span></label>
                <input type="text" id="client" name="client" placeholder="Nom du client ou raison sociale
                "
                onfocus="effacerAide(this)" onblur="verifierClient(this)">
                <span id="clientAide" class="aide"></span>
            </div>

            <div class="champ">
                <label for="produit">Produit <span style="color:red">*</span></label>
                <select id="produit" name="produit" onchange="majPrixUnitaire()">
                    <option value="">--- Sélectionner un produit ---</option>
                    <!-- Options injectées par JS dans initialiser() -->
                </select>
                <span id="produitAide" class="aide"></span>
            </div>

            <div class="champ">

```

```

        <label for="qte">Quantité <span style="color:red">*</span></label>
        <input type="text" id="qte" name="qte" placeholder="Ex : 3" onchange="calculerTotal()"
            inputmode="numeric" pattern="[0-9]*"> <!-- Clavier numérique mobile -->
        <span id="qteAide" class="aide"></span>
    </div>

    <div class="champ" style="background:#f8f9fa; padding:1rem; border-radius:4px; display:
flex; justify-content:space-between; align-items:center;">
        <span>Prix unitaire :</span>
        <output id="pu" name="pu">0</output> FCFA
    </div>

    <div class="champ" style="background:#e8f8f5; padding:1rem; border-radius:4px; border
:2px solid #27ae60; display:flex; justify-content:space-between; align-items:center;">
        <span style="font-size:1.1rem;">≡ <strong>Montant total :</strong></span>
        <output id="total" name="total">0</output> FCFA
    </div>

    <button type="submit">≡ Valider et Envoyer la commande</button>
</fieldset>
</form>
<p style="text-align:center; margin-top:1.5rem; color:#95a5a6; font-size:0.85rem;">
    Exercice 1ère TI — JavaScript Événements & Formulaire — © OnBuch+ \copyright \ <span id="
annee"></span>
</p>
</body>
</html>

```

LESAVAIS-TU ? Contrôle côté client vs côté serveur

Le JavaScript s'exécute dans le navigateur de l'utilisateur (côté client). L'utilisateur peut désactiver JavaScript dans les paramètres de son navigateur, ou un développeur malveillant peut modifier votre code via les outils de développement (F12) pour contourner vos contrôles. Jamais vous ne devez faire confiance aux données reçues du client sans les vérifier côté serveur (PHP, Node.js, Python, Java — programme de Terminale TI). Le contrôle côté client est un confort (réactivité, économie de requêtes), le contrôle côté serveur est une nécessité de sécurité (intégrité des données, protection contre injections SQL/XSS).

EXERCICE 7 Entraînement — Validation d'un formulaire d'inscription

Réaliser une page `inscription.html` contenant un formulaire avec les champs :

1. `<input id="nom" type="text">` (obligatoire, au moins 2 caractères).
2. `<input id="email" type="email">` (obligatoire, doit contenir "@" et "." après l'@ — test simple avec `indexOf` ou `includes`).

3. `<input id="age" type="text">` (obligatoire, entier entre 16 et 100).
4. `<select id="region">` avec les 10 régions du Cameroun (option `value=""` : "Choisir...").
5. Bouton `<button type="submit">S'inscrire</button>`.

Contraintes :

- Utiliser `onchange` sur `age` pour afficher « Mineur » / « Majeur » dans un `<span id="statutAge">` à côté du champ.
- Utiliser `onfocus` sur chaque champ pour effacer les messages d'erreur.
- Validation globale sur `onsubmit` : bloquer l'envoi (`return false`) et `alert()` listant tous les champs invalides si erreur ; sinon `alert("Inscription réussie !")` et `return true`.

CORRIGÉ Exercice Inscription

```
// Extrait des fonctions clés
function verifierAge() {
    const val = document.getElementById("age").value.trim();
    const statut = document.getElementById("statutAge");
    const age = parseInt(val, 10);
    if (val === "") { statut.textContent = ""; return; }
    if (isNaN(age) || age < 16 || age > 100 || val !== String(age)) {
        statut.textContent = "⚠️ Âge invalide (16-100)";
        statut.style.color = "red";
    } else {
        statut.textContent = (age >= 18) ? "⚠️ Majeur" : "⚠️ Mineur (autorisation parentale requise)";
        statut.style.color = (age >= 18) ? "green" : "orange";
    }
}

function verifierFormulaire() {
    const erreurs = [];
    const nom = document.getElementById("nom").value.trim();
    const email = document.getElementById("email").value.trim();
    const ageVal = document.getElementById("age").value.trim();
    const region = document.getElementById("region").value;

    if (nom.length < 2) erreurs.push("• Nom : au moins 2 caractères.");
    if (!email.includes("@") || !email.includes(".") || email.indexOf("@") > email.lastIndexOf("."))
        erreurs.push("• Email : format invalide (ex : user@domaine.cm).");

    const age = parseInt(ageVal, 10);
    if (ageVal === "" || isNaN(age) || age < 16 || age > 100 || ageVal !== String(age))
        erreurs.push("• Âge : entier entre 16 et 100 requis.");

    if (region === "") erreurs.push("• Région : choix obligatoire.");
}
```

```
if (erreurs.length > 0) {  
    alert("FORMULAIRE INVALIDE :\n\n" + erreurs.join("\n"));  
    return false;  
}  
alert("Inscription réussie pour " + nom + " (" + region + ") !");  
return true;  
}
```

À RETENIR**Synthèse : Évènements & Formulaires**

- Évènements clés : onclick, onmouseover, onchange, onfocus, onload (sur <body>).
- Association : Attribut HTML `onevent="maFonction(this)"` — `this` passe l'élément DOM.
- Accès DOM : `document.getElementById("id")` (guillemets!) → objet élément.
- Valeur champ : `element.value` toujours chaîne → `parseInt()`, `parseFloat()`.
- Validation : 4 étapes (Récupérer, Lire, Tester, Réagir). Bloquer envoi : `<form onsubmit="return verif()">+return false/true`.
- Sécurité : Contrôle client = confort (UX). Contrôle serveur = sécurité (obligatoire en Terminale).

7 ACTIVITÉ D'INTÉGRATION

7.1 Objectifs de l'activité

À l'issue de cette activité d'intégration, tu devras être capable de :

- créer un **formulaire HTML** complet et l'associer à un **script JavaScript** placé dans un **fichier externe** ;
- déclarer et utiliser des **variables**, des **constantes** et un **tableau** en JavaScript ;
- lire et convertir les données saisies grâce à `getElementById()`, `parseFloat()` et `parseInt()` ;
- contrôler la saisie avec les structures alternatives `if...else` et les opérateurs de comparaison et logiques ;
- calculer un montant avec remise conditionnelle à l'aide des opérateurs arithmétiques ;
- parcourir un tableau avec une boucle `for` pour calculer une somme et rechercher un maximum ;
- écrire une **fonction** qui retourne une valeur et une fonction qui n'en retourne pas ;
- exploiter les événements `onload` et `onclick` pour rendre la page réellement dynamique ;
- tester, déboguer et présenter une page web fonctionnelle dans un navigateur.

7.2 Situation

Monsieur Etoundi est le gérant du cyber-café « Njangui Net », situé au quartier Nlongkak à Yaoundé, non loin du lycée. Chaque jour, une cinquantaine de clients — élèves, étudiants, commerçants — viennent se connecter à Internet. Jusqu'ici, la facturation se fait à la main, sur un cahier : le caissier multiplie le nombre d'heures par le tarif horaire, ce qui provoque de fréquentes erreurs de calcul et des contestations de clients.

Pour fidéliser sa clientèle, M. Etoundi applique une règle commerciale simple : **toute connexion strictement supérieure à 5 heures donne droit à une remise de 10 % sur le montant total**. Le tarif horaire standard est de **500 FCFA**, mais il peut être modifié (par exemple 300 FCFA le samedi, journée promotionnelle).

Fatigué des erreurs, M. Etoundi sollicite ta classe de Première TI. Il souhaite une **page web de facturation interactive** fonctionnant dans le navigateur de son ordinateur de caisse, sans connexion Internet (tout se passe côté client). La page doit :

1. afficher un message de bienvenue au chargement de la page ;
2. proposer un formulaire avec trois champs : le **nom du client**, le **nombre d'heures** de connexion et le **tarif horaire** (pré-rempli à 500) ;
3. au clic sur le bouton « Calculer », vérifier que le nom n'est pas vide et que le nombre d'heures est un nombre strictement positif ; en cas d'erreur, afficher un message d'alerte explicite ;
4. calculer le montant de la facture, en appliquant la remise de 10 % si la durée dépasse 5 heures, et afficher le résultat dans la page ;

5. mémoriser le montant de chaque facture de la journée dans un tableau ;
6. au clic sur le bouton « Bilan », afficher le **nombre de factures**, la **recette totale** de la journée et le **montant de la plus grande facture**.

Par groupes de trois ou quatre, vous allez concevoir cette page, la tester dans le navigateur, puis la présenter à la classe. M. Etoundi choisira la meilleure réalisation pour son cyber-café !

7.3 Consignes

1. Crée un dossier `njangui_net` contenant deux fichiers : `facture.html` et `script.js`. Pourquoi est-il préférable de placer le script dans un fichier externe plutôt que dans le corps de la page ?
2. Dans `facture.html`, construis le formulaire : trois zones de texte identifiées par `nom`, `heures` et `tarif` (ce dernier avec la valeur par défaut 500), deux boutons (« Calculer » et « Bilan ») et une zone d'affichage des résultats. N'oublie pas la balise qui lie le fichier JavaScript externe.
3. Dans `script.js`, déclare une constante `SEUIL_REMISE` valant 5, une constante `TAUX_REMISE` valant 0,10 et un tableau vide `factures`. Justifie l'usage de `const` et de `let`.
4. Écris la fonction `bienvenue()` qui affiche, au chargement de la page, un message de bienvenue avec `alert()`. Quel événement et quel attribut HTML utilises-tu pour la déclencher ?
5. Écris la fonction `calculerFacture()` qui : lit les trois champs avec `document.getElementById(...).value` ; convertit les valeurs numériques avec `parseFloat()` ; vérifie la saisie (`nom` non vide, `heures` > 0, `tarif` > 0) ; calcule le montant avec remise éventuelle ; affiche le résultat ; ajoute le montant au tableau avec `push()`.
6. Écris la fonction `calculerBilan()` qui parcourt le tableau `factures` avec une boucle `for` et retourne la recette totale, puis la fonction `afficherBilan()` qui recherche la facture maximale et affiche le bilan complet.
7. Teste ta page avec les données suivantes et note les résultats obtenus : (a) Amina, 3 heures, 500 FCFA ; (b) Boris, 8 heures, 500 FCFA ; (c) un nom vide ; (d) un nombre d'heures égal à -2. Vérifie ensuite le bilan.
8. Présente ta page à la classe : montre le code, explique le rôle de chaque fonction et décris un bug rencontré pendant le développement et la façon dont tu l'as corrigé.

7.4 Ressources à mobiliser

À RETENIR

Boîte à outils de l'activité

- Insertion d'un script externe : `<script src="script.js"></script>` (placé en fin de `<body>`).
- Accès à un élément : `document.getElementById("id").value` (lecture) et `.innerHTML` (écriture dans la page).

- **Conversions** : `parseFloat("3,5".replace(",", "."))` ou `parseFloat("3.5")` donne 3,5; `parseInt("8")` donne 8 (utilise `parseInt` pour des nombres entiers, `parseFloat` pour des durées ou prix décimaux).
- **Montant avec remise** : si $h > 5$, alors $M = h \times t \times (1 - 0,10)$, sinon $M = h \times t$.
- **Tableau** : déclaration `let factures = []`; ajout `factures.push(m)`; taille `factures.length`.
- **Recherche du maximum** : initialiser `max` au premier élément, puis comparer chaque élément dans une boucle `for`.
- **Évènements HTML** : attribut `onload` sur `<body>`, attribut `onclick` sur les boutons (minuscules).

7.5 Démarche et corrigé commenté

EXERCICE RÉSOLU 7 Page de facturation du cyber-café « Njangui Net »

Énoncé : réaliser la page de facturation interactive décrite dans la situation, en deux fichiers, puis la tester avec les jeux de données de la consigne 7.

SOLUTION

Étape 1 — Structure HTML (`facture.html`). Le formulaire contient les trois champs identifiés, les deux boutons munis de l'évènement `onclick`, et une division `resultat` pour l'affichage. L'évènement `onload` du corps de page déclenche le message de bienvenue. Le script externe est chargé en fin de `<body>` pour garantir que le DOM est prêt.

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="UTF-8">
    <title>Njangui Net – Facturation</title>
  </head>
  <body onload="bienvenue()">
    <h1>Cyber-café Njangui Net – Facture client</h1>

    <label>Nom du client :</label>
    <input type="text" id="nom"><br><br>

    <label>Nombre d'heures :</label>
    <input type="text" id="heures"><br><br>

    <label>Tarif horaire (FCFA) :</label>
    <input type="text" id="tarif" value="500"><br><br>

    <button onclick="calculerFacture()">Calculer</button>
```

```
<button onclick="afficherBilan()">Bilan</button>

<div id="resultat"></div>

<!-- Liaison du script externe en fin de body -->
<script src="script.js"></script>
</body>
</html>
```

Étape 2 — Le script externe (script.js). Les constantes (const) protègent les valeurs de référence (seuil, taux) contre toute réaffectation accidentelle ; let est utilisé pour le tableau factures car sa contenu change (ajout d'éléments) mais pas sa référence, et pour les variables locales des fonctions. C'est la bonne pratique moderne (ES6+).

```
// Constantes de l'application (valeurs fixes, lisibles, maintenables)
const SEUIL_REMISE = 5; // seuil en heures (nombre entier)
const TAUX_REMISE = 0.10; // remise de 10 % (nombre décimal)

// Tableau des montants de la journée : let car on y ajoute des éléments (push)
let factures = [];

// Fonction SANS retour : message au chargement (événement onload)
function bienvenue() {
    alert("Bienvenue au cyber-café Njangui Net !");
}

// Fonction SANS retour : calcule, affiche et mémorise une facture
function calculerFacture() {
    // 1. Lecture des champs (retourne des chaînes de caractères)
    let nom = document.getElementById("nom").value;
    let heures = parseFloat(document.getElementById("heures").value);
    let tarif = parseFloat(document.getElementById("tarif").value);

    // 2. Contrôle de saisie rigoureux
    if (nom === "") {
        alert("Erreur : veuillez saisir le nom du client.");
    } else if (isNaN(heures) || heures <= 0) {
        alert("Erreur : le nombre d'heures doit être un nombre positif.");
    } else if (isNaN(tarif) || tarif <= 0) {
        alert("Erreur : le tarif horaire doit être un nombre positif.");
    } else {
        // 3. Calcul du montant
        let montant = heures * tarif;
        let message = "Client : " + nom + "\n";

        // Application de la remise conditionnelle
        if (heures > SEUIL_REMISE) {
            montant = montant * (1 - TAUX_REMISE);
            message = message + "Remise de 10 % appliquée.\n";
        }

        // 4. Mémorisation et affichage
```

```
factures.push(montant); // Ajout au tableau de la journée
message = message + "Montant à payer : " + montant.toFixed(0) + " FCFA";

// Remplacement des retours à la ligne par <br> pour l'affichage HTML
document.getElementById("resultat").innerHTML =
    "<p>" + message.replace(/\n/g, "<br>") + "</p>";
}
}

// Fonction AVEC retour : calcule la recette totale (parcours tableau)
function calculerBilan() {
    let total = 0;
    for (let i = 0; i < factures.length; i++){
        total = total + factures[i];
    }
    return total; // Retourne la somme pour réutilisation
}

// Fonction SANS retour : affiche le bilan complet (nb, total, max)
function afficherBilan() {
    if (factures.length === 0) {
        alert("Aucune facture enregistrée aujourd'hui.");
    } else {
        let total = calculerBilan(); // Réutilisation de la fonction
        let max = factures[0]; // Initialisation du maximum

        // Recherche du maximum (parcours à partir du 2e élément)
        for (let i = 1; i < factures.length; i++){
            if (factures[i] > max) {
                max = factures[i];
            }
        }

        document.getElementById("resultat").innerHTML =
            "<p>Bilan du jour :<br>" +
            "Nombre de factures : " + factures.length + "<br>" +
            "Recette totale : " + total.toFixed(0) + " FCFA<br>" +
            "Plus grande facture : " + max.toFixed(0) + " FCFA</p>";
    }
}
```

Étape 3 — Vérification des jeux d'essai.

- Amina, 3 h à 500 FCFA : $3 \times 500 = 1500$ FCFA (pas de remise, car $3 \leq 5$).
- Boris, 8 h à 500 FCFA : $8 \times 500 = 4000$, puis remise : $4000 \times 0,9 = 3600$ FCFA.
- Nom vide : message d'alerte « Erreur : veuillez saisir le nom du client. », aucun calcul effectué.
- Heures = -2 : message d'alerte « Erreur : le nombre d'heures doit être un nombre positif. », aucun calcul effectué.
- Bilan : 2 factures, recette totale $1500 + 3600 = 5100$ FCFA, facture maximale 3600

FCFA.

Les résultats affichés par la page correspondent exactement aux calculs manuels : la page est validée.

ATTENTION Pièges fréquents rencontrés pendant l'activité

- Oublier `parseFloat()` / `parseInt()` : la valeur lue par `.value` est une chaîne ; `"3" + "500"` donne `"3500"` (concaténation) et non `3500` (addition).
- Virgule décimale : `parseFloat("2,5")` renvoie `2` (arrêt au premier caractère invalide) ! Saisir les durées avec un point (`2.5`) ou remplacer la virgule avant conversion : `parseFloat(valeur.replace(",", "."))`.
- Mauvaise casse : `getElementById` est sensible à la casse ; `getElementById(D majuscule)` provoque une erreur `Uncaught TypeError`.
- Script chargé trop tôt : si la balise `<script src="...">` est placée dans `<head>` sans attribut `defer`, les éléments HTML n'existent pas encore au moment de l'exécution ; placer la balise en fin de `<body>` (comme ici) ou utiliser `defer`.
- Confondre `=` et `===` dans les tests : `if (nom = "")` affecte la chaîne vide à `nom` et retourne `""` (faux), au lieu de comparer. Toujours écrire `===` (égalité stricte) ou `==`.
- Variable `max` mal initialisée : initialiser `let max = 0` échoue si toutes les factures sont négatives (impossible ici) ou si le tableau est vide (géré par le `if` précédent). La méthode robuste est `let max = factures[0]` ; puis boucle à partir de l'indice `1`.

7.6 Bilan de l'activité

Cette activité a permis de mobiliser, dans un projet concret et complet, l'ensemble des savoir-faire de la leçon : insertion d'un script externe dans une page HTML, déclaration de variables, constantes et tableaux, conversions avec `parseFloat()` et `parseInt()`, contrôle de saisie par `if...else` et opérateurs logiques, calculs arithmétiques avec remise conditionnelle, fonctions avec et sans valeur de retour, parcours de tableau par boucle `for`, recherche d'un maximum, et gestion des événements `onload` et `onclick`.

Au-delà de la technique, tu as vécu la démarche réelle d'un développeur : analyser un besoin exprimé par un client (M. Etoundi), le traduire en règles de calcul précises, coder, tester avec des jeux d'essai choisis (y compris des saisies erronées), corriger les bugs et présenter le produit. Tu as aussi constaté qu'une page dynamique côté client fonctionne **sans serveur ni connexion Internet** : tout s'exécute dans le navigateur, ce qui est précieux pour une petite entreprise comme le cyber-café « Njangui Net ». Enfin, le travail en groupe et la présentation devant la classe développent des savoir-être essentiels : collaboration, rigueur, esprit critique et clarté dans la communication.

8 MÉTHODES ET EXERCICES RÉSOLUS

9 PROGRAMMATION JAVASCRIPT CÔTÉ CLIENT

DÉFINITION Script JavaScript — Côté client

Un script est une suite d'instructions écrites dans un langage de script (ici JavaScript) et destinées à être exécutées par un interpréteur intégré au navigateur web. En côté client (client-side), le code est téléchargé avec la page HTML et s'exécute sur la machine de l'utilisateur (ordinateur, téléphone), sans aller-retour vers le serveur. Cela permet de rendre l'interface homme-machine (IHM) réactive : validation instantanée de formulaires, animations, mise à jour partielle de la page (DOM), calculs locaux.

À RETENIR Avantages et limites de JavaScript côté client

- Avantages : Réactivité immédiate (pas de latence réseau) ; allègement du serveur (traitement local) ; enrichissement de l'IHM (drag-and-drop, carrousels, cartes interactives) ; langage universel (tous navigateurs modernes).
- Limites : Sécurité : le code est visible et modifiable par l'utilisateur (outils développeur) → jamais de confiance aveugle pour des opérations critiques (paiement, droits d'accès, BDD) ; Dépendance navigateur : différences mineures d'API (rares aujourd'hui) ; Accès restreint : pas d'accès direct au système de fichiers local, à la base de données du serveur, ni aux périphériques sans autorisation explicite (Webcam, GPS via API sécurisées).

9.1 Scripts, événements et insertion de JavaScript dans HTML

DÉFINITION Évènement

Un évènement est une action détectée par le navigateur sur un élément HTML : clic de souris (`click`), survol (`mouseover`), frappe clavier (`keydown`), chargement de la page (`load`), modification d'un champ (`change`), prise de focus (`focus`). JavaScript permet d'associer une fonction gestionnaire (`handler`) à cet évènement pour y réagir.

MÉTHODE Bien insérer un script : les trois méthodes

Pour intégrer du JavaScript dans une page HTML, procède ainsi :

1. Choisir l'emplacement :

- Dans l'en-tête `<head>` : idéal pour déclarer des fonctions qui seront appelées plus tard (événements, `onload`). Le script est lu avant l'affichage du `<body>`.
- Dans le corps `<body>` : le script s'exécute au moment où le navigateur le rencontre lors du parsing HTML. Utile pour écrire du contenu dynamique au chargement (`document.write`), mais bloque l'affichage.
- Dans un fichier externe (extension `.js`) : méthode recommandée (propre, réutilisable, mise en cache navigateur).

2. Script interne : encadrer le code par les balises `<script>` et `</script>`.

```
<script>  
  // Code JavaScript ici  
  console.log("Script interne chargé");  
</script>
```

3. Script externe : utiliser l'attribut `src` sur une balise `<script>` non auto-fermante.

```
<script src="mesScripts/fonctions.js"></script>
```

Interdiction : on ne met jamais de code JavaScript entre les balises d'un `<script src="...">` (il serait ignoré).

4. Attribut `defer` / `async` (bonus) : dans le `<head>`, `<script src="..." defer></script>` attend la construction du DOM avant d'exécuter (équivalent à placer le script en fin de `<body>`). `async` exécute dès que possible (ordre non garanti).

Réflexe Probatoire : si l'énoncé demande « un script réutilisable sur plusieurs pages » ou « séparation structure/comportement », c'est le fichier externe (`.js`) qu'il faut citer.

EXERCICE RÉSOLU 8 Insertion d'un script dans une page

Énoncé. La boutique « Chez Manga » de Douala veut afficher un message de bienvenue à l'ouverture de sa page d'accueil.

1. Écrire le code HTML complet d'une page contenant, dans l'en-tête, un script qui affiche la boîte de dialogue « Bienvenue chez Manga! ».
2. Réécrire uniquement la ligne d'insertion si le script est placé dans un fichier externe nommé `bienvenue.js`.

SOLUTION

Solution.

1. Le script est placé entre `<head>` et `</head>`, encadré par la balise `<script>`. La fonction `alert()` affiche une boîte de dialogue modale.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Boutique Chez Manga</title>
  <script>
    // Script interne place dans l'en-tete
    alert("Bienvenue chez Manga !");
  </script>
</head>
<body>
  <h1>Boutique Chez Manga</h1>
  <p>Vos articles scolaires au meilleur prix.</p>
</body>
</html>
```

2. Pour un fichier externe, on utilise l'attribut `src` et on laisse le corps de la balise vide :

```
<script src="bienvenue.js"></script>
```

Le fichier `bienvenue.js` contient alors uniquement la ligne `alert("Bienvenue chez Manga !");`, sans balise `<script>`.

Conclusion : un script interne s'écrit entre `<script>` et `</script>`, un script externe se lie avec `<script src="fichier.js"></script>`.

9.2 Variables, constantes, types et opérateurs

MÉTHODE Déclarer et calculer sans se tromper

1. Déclarer : `const` pour une valeur qui ne change pas (référence constante), `let` pour une variable réaffectable. Éviter `var` (portée fonction, hoisting source de bugs).

```
const TVA = 19.25; // Constante : séparateur DECIMAL = POINT
let quantite = 10; // Variable modifiable
let nomClient;    // Declaration sans initialisation (undefined)
```

2. Types de base (primitifs) : `number` (entiers, flottants, `NaN`, `Infinity`), `string` (chaîne : guillemets simples `'...'`, doubles `"..."` ou backticks ``...``), `boolean` (`true/false`),

undefined, null. JavaScript est à typage dynamique : le type suit la valeur.

3. Opérateurs :

- **Arithmétiques** : +, -, *, /, % (modulo, reste), ** (puissance).
- **Comparaison** : === (égalité stricte : valeur ET type), !== (différence stricte), >, >=, <, <=. Proscrivez == et != (coercition de type imprévisible : 0 == "0" // true).
- **Logiques** : && (ET), || (OU), ! (NON). Court-circuit : expr1 && expr2 **évalue** expr2 seulement si expr1 est truthy.
- **Assignment** : =, +=, -=, *=, /=, %=.
- **Concaténation** : + entre chaînes. Piège : "5" + 3 → "53" (chaîne), 5 + 3 → 8 (nombre). Si un opérande est chaîne, + concatène.

4. Tester mentalement chaque ligne avec des valeurs concrètes avant de conclure.

EXERCICE RÉSOLU 9 Trace d'exécution et opérateurs

Énoncé. On donne le script suivant :

```
let a = 10, b = 4, c = "10";  
let r1 = a % b;  
let r2 = a / b;  
let r3 = c + b;  
let r4 = (a > b) && (b > 0);  
a += 5;
```

Donner la valeur et le type de r1, r2, r3, r4, puis la valeur finale de a.

SOLUTION

Solution. On exécute le script ligne par ligne :

- $r1 = a \% b$: l'opérateur % donne le reste de la division entière. Or $10 = 4 \times 2 + 2$, donc $r1 = 2$ (type number).
- $r2 = a / b$: division réelle : $10/4 = 2,5$, donc $r2 = 2.5$ (type number).
- $r3 = c + b$: c est la chaîne "10" (guillemets!). L'opérateur + concatène : $r3 = "104"$ (type string).
- $r4 = (a > b) \&\& (b > 0)$: $10 > 4$ est vrai, $4 > 0$ est vrai, donc vrai ET vrai = vrai : $r4 = true$ (type boolean).
- $a += 5$ équivaut à $a = a + 5$, donc $a = 10 + 5 = 15$.

Conclusion : $r1 = 2, r2 = 2.5, r3 = "104", r4 = true$ et $a = 15$. La présence de guillemets transforme un nombre en chaîne et change totalement le sens de +.

9.3 Instructions simples : entrées, sorties et conversions

MÉTHODE Lire, convertir, afficher

1. **Entrée (boîte de dialogue)** : `let saisie = prompt("Message :");` renvoie toujours une string (ou null si Annuler). Même si l'utilisateur tape 12, `saisie` vaut "12".
2. **Conversion OBLIGATOIRE avant tout calcul** :
 - `parseInt(chaine [, base])` : convertit en entier (base 10 par défaut).
`parseInt("12.5") → 12`, `parseInt("abc") → NaN`.
 - `parseFloat(chaine)` : convertit en flottant. `parseFloat("12.5") → 12.5`.
 - `Number(chaine)` : conversion stricte (`Number("12.5") → 12.5`, `Number("abc") → NaN`).

Usage type : `let age = parseInt(prompt("Age ?"));`

3. **Sorties** :
 - `alert(message)` : boîte modale (bloquante). Débogage simple / utilisateur.
 - `document.write(texte)` : écrit dans le flux HTML au chargement. Attention : après chargement complet (onload), écrase toute la page. Réservé aux exercices simples.
 - `console.log(valeur)` : console développeur (F12). Outil pro n°1 pour déboguer (objets, tableaux, traces non bloquantes).
 - `element.textContent = ... / innerHTML = ...` : écriture dans le DOM (vrai dynamisme, vu plus loin).
4. **Construire le message par concaténation (+) ou template literals (ES6)** : ``Total : $total FCFA``.
5. **Vérifier** : si le résultat d'un calcul donne des chiffres collés (ex. 53 au lieu de 8), c'est qu'une conversion `parseInt/parseFloat` a été oubliée.

EXERCICE RÉSOLU 10 Calcul d'une moyenne de notes

Énoncé. Un enseignant du lycée de Nkongsamba veut un script qui demande les notes de deux épreuves de TI (sur 20) et affiche la moyenne. Écrire ce script en JavaScript.

SOLUTION

Solution. Les valeurs saisies via `prompt` sont des chaînes : on les convertit avec `parseFloat` car une note peut être décimale (ex. 12,5). La moyenne est $m = \frac{n_1 + n_2}{2}$.

```
// Saisie des deux notes avec conversion immédiate
const note1 = parseFloat(prompt("Entrez la note 1 :"));
const note2 = parseFloat(prompt("Entrez la note 2 :"));

// Calcul de la moyenne : parenthèses OBLIGATOIRES !
const moyenne = (note1 + note2) / 2;

// Affichage du résultat (template literal ES6)
alert('La moyenne est : ${moyenne} / 20');
```

Vérification : pour $\text{note1} = 12$ et $\text{note2} = 15$, on obtient $m = \frac{12 + 15}{2} = \frac{27}{2} = 13,5$. Le message affiché est « La moyenne est : 13.5 / 20 ».

Sans `parseFloat`, $\text{note1} + \text{note2}$ vaudrait la chaîne "1215" et le résultat serait absurde. Sans parenthèses, $\text{note1} + \text{note2} / 2$ calculerait $12 + 7,5 = 19,5$ (priorité de la division).

Conclusion : toute donnée issue de `prompt` (ou `.value`) doit être convertie avant calcul, et la formule de moyenne exige des parenthèses autour de la somme.

9.4 Structures alternatives et boucles

MÉTHODE Choisir et rédiger une structure de contrôle

1. Alternative simple : `if (condition) { ... };` avec deux cas : `if (condition) { ... } else { ... };`. La condition est une expression boolean. Jamais = seul dans un test (affectation); on utilise `===` (strict) ou `==` (toléré mais déconseillé).
2. Cas multiples : enchaîner `if ... else if ... else`.
3. Boucle `for` : quand le nombre de répétitions est connu ou pour parcourir un tableau (indices). `for (let i = 0; i < n; i++) { ... }` (initialisation; condition; incrément).
4. Boucle `while` : tant qu'une condition est vraie (test en entrée, corps peut ne jamais s'exécuter). `while (condition) { ... };`
5. Boucle `do...while` : le corps s'exécute au moins une fois (test en sortie). Point-virgule final obligatoire. `do { ... } while (condition);`
6. Garantir l'arrêt : vérifier que la variable de contrôle est modifiée dans le corps vers la condition d'arrêt, sinon boucle infinie (gel navigateur).
7. `break` / `continue` : `break` sort de la boucle; `continue` passe à l'itération suivante.

EXERCICE RÉSOLU 11 Mention d'un élève et table de multiplication

Énoncé.

1. Écrire un script qui demande la moyenne d'un élève et affiche « Admis » si elle est supérieure ou égale à 10, « Ajourné » sinon.
2. Écrire une boucle `for` qui affiche la table de multiplication de 7 (de 7×1 à 7×10) dans la page.

SOLUTION

Solution.

1. La condition porte sur la moyenne convertie en nombre :

```
const moy = parseFloat(prompt("Entrez la moyenne :"));
if (moy >= 10) {
    alert("Admis");
} else {
    alert("Ajourné");
}
```

Le test `moy >= 10` couvre le cas limite 10/20 : l'élève est Admis, conformément à la règle.

2. Le nombre de tours est connu (10) : la boucle `for` est adaptée.

```
for (let i = 1; i <= 10; i++) {
    document.write(`7 x ${i} = ${7 * i}<br>`);
}
```

Trace : `i` vaut successivement 1, 2, ..., 10 ; à `i = 11`, la condition `i <= 10` devient fausse et la boucle s'arrête. La page affiche « 7 x 1 = 7 » jusqu'à « 7 x 10 = 70 ». La balise `<br>` provoque le retour à la ligne HTML.

Conclusion : `if...else` traite une décision à deux branches ; la boucle `for (init ; cond ; inc)` est idéale pour un nombre d'itérations connu.

9.5 Tableaux, objets et fonctions

MÉTHODE Manipuler tableaux et fonctions

1. Déclarer un tableau : littéral `const notes = [12, 15, 9]` ; (recommandé) ou `new Array(12, 15, 9)` ; `new Array(3)` crée un tableau vide de longueur 3 (piège!).
2. Indices : base 0. Premier : `tab[0]`, dernier : `tab[tab.length - 1]`. `length` = nombre d'éléments (propriété, pas méthode).
3. Parcours standard :

```
for (let i = 0; i < tab.length; i++){  
  console.log(tab[i]);  
}
```

Parcours `for...of` (ES6, valeurs): `for (const val of tab) { ... }.`

4. Recherche : parcourir en comparant chaque élément à la valeur cherchée (retourner `true/false` ou l'indice).

5. Objets : collection de paires clé: valeur.

- **Littéral** : `const eleve = {nom: "Amina", age: 16};` (**accès** `eleve.nom` ou `eleve["nom"]`).
- **Constructeur natif**: `const d = new Date();` (**méthodes** `getFullYear()`, `getTime()`, etc.).

6. Fonction : bloc réutilisable.

```
function nom(param1, param2){  
  // instructions  
  return resultat; // optionnel  
}
```

Return : termine la fonction et renvoie une valeur. Sans `return` (ou `return;`), la fonction renvoie `undefined`.

7. Appel: `const res = nom(arg1, arg2);`.

EXERCICE RÉSOLU 12 Maximum d'un tableau de notes

Énoncé. Les notes de TI d'une classe sont stockées dans un tableau. Écrire une fonction `noteMax(tab)` qui reçoit un tableau de nombres et retourne la plus grande note, puis un appel avec le tableau `[8, 14, 11, 17, 10]`.

SOLUTION

Solution. On applique l'algorithme classique de recherche de maximum : on initialise le maximum au premier élément, puis on parcourt le reste du tableau en mettant à jour le maximum dès qu'on trouve plus grand.

```
function noteMax(tab){  
  if (tab.length === 0) return undefined; // Garde-fou  
  let max = tab[0]; // maximum provisoire  
  for (let i = 1; i < tab.length; i++){  
    if (tab[i] > max){  
      max = tab[i]; // nouveau maximum trouve  
    }  
  }  
}
```

```

    }
    return max;           // la fonction retourne une valeur
}

```

```

// Appel de la fonction
const notes = [8, 14, 11, 17, 10];
alert('La meilleure note est : ${noteMax(notes)}');

```

Trace détaillée : $\text{max} = 8$; $i=1: 14 > 8 \rightarrow \text{max} = 14$; $i=2: 11 < 14$ (inchangé); $i=3: 17 > 14 \rightarrow \text{max} = 17$; $i=4: 10 < 17$ (inchangé). Boucle finie ($i = 5 = \text{tab.length}$).

Conclusion : la fonction retourne 17, et le message « La meilleure note est : 17 » s'affiche. Sans `return`, l'appelant ne pourrait pas récupérer le résultat.

9.6 Évènements et contrôle des formulaires

MÉTHODE Réagir aux évènements et valider une saisie

1. Associer un évènement (niveau Probatoire : attribut HTML inline) :

- `onClick` (clic souris) sur bouton, lien, div...
- `onMouseOver` / `onMouseOut` (survol / sortie souris).
- `onChange` (perte de focus après modification) sur `<input>`, `<select>`.
- `onFocus` / `onBlur` (prise / perte de focus) sur champ.
- `onLoad` sur `<body>` (page + ressources chargées).
- `onSubmit` sur `<form>` (validation formulaire, `return false` pour bloquer).

Valeur = appel de fonction : `onClick="verifier()"`.

2. Accéder à un élément HTML par son id unique : `const elt = document.getElementById("id");`.

- Valeur d'un champ (`input`, `select`, `textarea`) : `elt.value` (toujours string).
- Contenu HTML : `elt.innerHTML` ou `elt.textContent`.
- Modifier style : `elt.style.color = "red";`.

3. Convertir la valeur lue (string) avec `parseInt` / `parseFloat` / `Number` si calcul.

4. Contrôler (Validation) :

- Champ vide : `if (valeur === "")` ou `if (!valeur.trim())`.
- Intervalle : `if (age < 0 || age > 120)`.
- Format (email, téléphone) : expressions régulières (aperçu) ou `input type="email"` (HTML5).

Afficher erreur : `alert(...)` ou injection DOM (`span erreur`).

5. Tester avec valeurs limites : vide, 0, borne min, borne max, texte au lieu de nombre, espaces.

EXERCICE RÉSOLU 13 Validation d'un formulaire d'inscription

Énoncé. Un cybercafé de Yaoundé propose un formulaire avec un champ de saisie d'identifiant "age" et un bouton « Valider ». Écrire le code HTML et JavaScript complet : au clic sur le bouton (événement `onClick`), une fonction `verifier()` lit l'âge saisi et affiche « Inscription acceptée » si l'âge est supérieur ou égal à 15 ans, « Âge insuffisant » sinon, et « Veuillez saisir votre âge » si le champ est vide.

SOLUTION

Solution. On suit la démarche : champ avec `id`, bouton avec `onClick`, fonction de contrôle utilisant `document.getElementById`.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Inscription cybercafe</title>
  <script>
    function verifier() {
      // 1. Recuperation de l'element et de sa valeur (string)
      const champ = document.getElementById("age");
      const saisie = champ.value.trim(); // trim() supprime espaces debut/fin

      // 2. Controle saisie vide
      if (saisie === "") {
        alert("Veuillez saisir votre age");
        return; // Arrete la fonction ici
      }

      // 3. Conversion en nombre (entier)
      const age = parseInt(saisie, 10); // Base 10 explicite

      // 4. Controle logique (NaN si saisie non numerique)
      if (isNaN(age)) {
        alert("Veuillez entrer un nombre valide");
      } else if (age >= 15) {
        alert("Inscription acceptée");
      } else {
        alert("Âge insuffisant");
      }
    }
  </script>
</head>
```

```
<body>
  <h1>Formulaire d'inscription</h1>
  <form action="#" onsubmit="return false;" > <!-- Empeche envoi reel pour l'exercice -->
    <label for="age">Age : </label>
    <input type="text" id="age" name="age">
    <input type="button" value="Valider" onclick="verifier()">
  </form>
</body>
</html>
```

Justification :

- `document.getElementById("age")` sélectionne l'unique élément `id="age"`.
- `.value` récupère le texte saisi (`string`). `.trim()` nettoie les espaces.
- `Test saisie === ""` avant conversion (sinon `parseInt("")` → `NaN`).
- `parseInt(saisie, 10)` : base 10 forcée (évite octal sur vieux navigateurs si "010").
- `isNaN(age)` détecte si la conversion a échoué (ex. saisie "abc").
- `onclick="verifier()"` (minuscules recommandées en HTML5) lie l'évènement clic.

Vérification : champ vide → « Veuillez saisir votre âge » ; saisie "abc" → « Veuillez entrer un nombre valide » ; saisie 17 → « Inscription acceptée » ; saisie 12 → « Âge insuffisant ».

Conclusion : le contrôle de saisie côté client repose sur trois réflexes : récupérer (`getElementById(...).value`), tester (vide, `NaN`, bornes), convertir (`parseInt/parseFloat`) avant tout calcul ou comparaison numérique.

ATTENTION Les 5 erreurs qui coûtent des points au Probatoire

1. Confondre `=` et `===` : `if (moy = 10)` affecte 10 à `moy` (valeur 10, `truthy`) au lieu de tester ; la condition est alors toujours vraie. Dans un test, on écrit `===` (strict) ou `==` (toléré).
2. Oublier `parseInt/parseFloat/Number` après prompt ou `.value` : "12" + "15" donne "1215" (concaténation) et non 27. Toute saisie utilisateur est une `string` tant qu'elle n'est pas convertie.
3. Mal indexer les tableaux : le premier élément est `tab[0]`, le dernier `tab[tab.length - 1]`. Écrire `i <= tab.length` dans la condition de boucle provoque un accès `undefined` (débordement).
4. Oublier le `return` dans une fonction censée retourner une valeur : l'appelant reçoit `undefined`. Distinguer clairement fonction « action » (effet de bord : affichage, modification DOM, sans `return`) et fonction « calcul » (valeur de retour, avec `return`).
5. Mal écrire l'insertion du script : oublier le `</script>` de fermeture, écrire `<script src="fichier.js" />` (auto-fermante interdite), ou mettre du code interne dans une balise qui possède déjà un `src`. Autre piège classique : écrire les décimaux avec une

virgule (13,5) au lieu du point (13.5) dans le code JS.

LESAVAIS-TU ? JavaScript au Cameroun : de la cybercafé au Mobile Money

Dans les années 2000, les cybercafés de Yaoundé, Douala ou Bafoussam étaient les premiers lieux d'exécution de JavaScript côté client pour le grand public : validation de formulaires d'inscription Yahoo/Hotmail, jeux Flash (ActionScript, cousin de JS), premiers chats AJAX. Aujourd'hui, avec l'explosion du Mobile Money (MTN MoMo, Orange Money) et des services en ligne de l'administration (e-visa, impôts, résultats d'examens sur www.minesec.gov.cm), JavaScript est le moteur invisible de l'inclusion numérique. Les frameworks modernes (React, Vue, Angular) utilisés par les startups de la Silicon Mountain (Buea) reposent tous sur les bases ES6+ que tu maîtrises ici : variables `let/const`, fonctions fléchées, promesses, manipulation du DOM. Ce cours est ta porte d'entrée vers le développement Full Stack camerounais de demain.

10 EXERCICES

10.1 Je vérifie mes connaissances

EXERCICE 8 QCM : premiers pas en JavaScript

Pour chaque question, une seule réponse est correcte. Recopie la lettre correspondante.

1. Pour insérer un script JavaScript directement dans une page HTML, on utilise la balise :
 - (a) `<js>`
 - (b) `<javascript>`
 - (c) `<script>`
 - (d) `<code>`
2. L'instruction qui affiche une boîte de dialogue contenant un message est :
 - (a) `prompt()`
 - (b) `alert()`
 - (c) `print()`
 - (d) `echo()`
3. En JavaScript, l'opérateur de comparaison d'égalité (égalité faible) est :
 - (a) `=`
 - (b) `==`
 - (c) `:=`
 - (d) `equals`
4. Soit la déclaration `var t = [5, 9, 2];`. La valeur de `t[1]` est :
 - (a) 5
 - (b) 9
 - (c) 2
 - (d) une erreur

EXERCICE 9 Vrai ou faux — justifie ta réponse

Indique si chacune des affirmations suivantes est vraie ou fausse, puis justifie brièvement.

1. JavaScript est un langage exécuté par le navigateur de l'internaute (côté client).
2. L'instruction `var age = prompt("Votre âge ?");` permet de faire directement des calculs arithmétiques sur `age` sans aucune précaution.
3. Une boucle `do...while` exécute son corps au moins une fois, même si la condition est fausse dès le départ.
4. L'évènement `onLoad` se déclenche lorsque l'utilisateur clique sur un bouton.

EXERCICE 10 Définitions et question de cours

1. Définis les termes suivants : **script** ; **évènement**.
2. Cite deux avantages et deux limites de JavaScript utilisé côté client.
3. Explique la différence entre `parseInt("7.9")` et `parseFloat("7.9")`, en précisant la valeur retournée par chacun.

EXERCICE 11 Évaluation d'expressions

Soient les déclarations suivantes :

```
var a = 10;  
var b = "3";  
var c = 4;
```

Donne la valeur (et le type si nécessaire) de chacune des expressions suivantes :

1. `a + c`
2. `a + b`
3. `a + parseInt(b)`
4. `a % c`
5. `(a > c) && (c < 5)`
6. `parseFloat("12,5".replace(",", "."))` (indique simplement le résultat attendu)

10.2 Je m'entraîne

EXERCICE 12 Table de multiplication

1. Écris un script JavaScript complet (à insérer dans une page HTML) qui demande à l'utilisateur un nombre entier à l'aide de `prompt()`, le convertit avec `parseInt()`, puis affiche à l'aide de `alert()` ou de `document.write()` la table de multiplication de ce nombre de 1 à 10, en utilisant une boucle `for`.
2. Réécris le même script en utilisant une boucle `while`.

EXERCICE 13 Gestion des ventes d'une boutique

Le gérant d'une boutique de Douala a enregistré ses ventes hebdomadaires (en FCFA) dans le tableau suivant :

```
var ventes = [15000, 8000, 22000, 12000];
```

1. Écris une fonction `totalVentes(t)` qui reçoit un tableau en paramètre et retourne la somme de tous ses éléments.
2. Écris une fonction `plusGrandeVente(t)` qui retourne la plus grande valeur du tableau.
3. Écris l'instruction qui affiche dans une boîte de dialogue le message : « Total : 57000 FCFA — Meilleure vente : 22000 FCFA » en utilisant les deux fonctions précédentes.

EXERCICE 14 Parité d'un nombre

Écris un script JavaScript qui :

1. demande à l'utilisateur de saisir un nombre entier avec `prompt()` ;
2. convertit la saisie en entier ;
3. affiche « Le nombre est pair » ou « Le nombre est impair » selon le cas (on utilisera l'opérateur `%` et une structure `if...else`).

EXERCICE 15 Premiers évènements

On considère la page HTML suivante :

```
<!DOCTYPE html>
<html>
<head><meta charset="utf-8"><title>Accueil</title></head>
<body>
  <p id="message">Survolez-moi !</p>
  <button>Cliquez-moi</button>
</body>
```

```
</html>
```

1. Modifie la balise `<body>` pour qu'un message « Bienvenue sur notre site ! » s'affiche au chargement de la page (événement `onLoad`).
2. Ajoute au paragraphe l'évènement `onMouseOver` de sorte que son texte devienne « Bravo, vous m'avez survolé ! » (on utilisera `document.getElementById`).
3. Ajoute au bouton l'évènement `onClick` qui déclenche l'affichage d'une boîte de dialogue « Merci pour votre clic ».

10.3 Je me prépare au Probatoire

EXERCICE 16 Exercice type Probatoire : chasse aux erreurs

Le site web du lycée bilingue de Bafoussam contient la page suivante, censée demander l'âge d'un élève et afficher s'il est majeur (âge ≥ 18) ou mineur. Malheureusement, le script contient cinq erreurs et ne fonctionne pas.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Test majorite</title>
</head>
<body>
  <script>
    var ages = [15, 19, 21];
    var saisie = prompt("Quel est votre age ?");
    var age = saisie;
    if (age = 18) {
      alert("Vous venez d'atteindre la majorite !");
    }
    if (age >= 18) {
      alert("Vous etes majeur. Premier age du tableau : " + ages[3]);
    } else {
      alert("Vous etes mineur.");
    }
    var zone = document.getElementById(resultat);
  </script>
</body>
</html>
```

1. Releve et corrige chacune des cinq erreurs en précisant à chaque fois : la ligne concernée, la nature de l'erreur (syntaxe, logique, conversion, accès au tableau, accès à un élément HTML) et la correction exacte à apporter.

2. Explique pourquoi la comparaison `if (age >= 18)` risque de donner un résultat faux même après correction de l'erreur de la question 1, tant que la saisie n'est pas convertie. Propose la correction définitive.
3. Après toutes les corrections, indique ce qu'affiche le script si l'utilisateur saisit la valeur 20.

EXERCICE 17 Exercice type Probatoire : mentions au Probatoire TI

Le conseil de classe d'un lycée de Yaoundé souhaite automatiser l'attribution des mentions. Le barème est le suivant :

Note n (sur 20)	Mention
$n < 10$	Ajourné
$10 \leq n < 12$	Passable
$12 \leq n < 16$	Bien
$n \geq 16$	Très bien

1. Écris en JavaScript une fonction `mention(n)` qui reçoit une note en paramètre et retourne la chaîne de caractères correspondant à la mention (on utilisera des structures `if...else if...else` et des opérateurs de comparaison et logiques).
2. Écris le script principal qui : demande la note à l'utilisateur avec `prompt()` ; la convertit en nombre décimal avec `parseFloat()` ; appelle la fonction `mention()` ; affiche par exemple « Note : 13,5 — Mention : Bien ».
3. Le délégué de classe propose de saisir les notes de 5 élèves et de les stocker dans un tableau déclaré avec `new Array()`. Écris ce script : une boucle `for` remplit le tableau, puis une seconde boucle affiche pour chaque élève sa note et sa mention.
4. Quelle mention obtient un élève ayant saisi la valeur "abc" si l'on ne contrôle pas la saisie ? Justifie ta réponse et propose un test à ajouter pour éviter ce problème.

EXERCICE 18 Exercice type Probatoire : formulaire d'inscription à un concours

L'université de Ngaoundéré met en ligne un formulaire d'inscription à un concours d'entrée. Les conditions sont les suivantes : le nom du candidat est obligatoire ; son âge doit être compris entre 17 et 25 ans (bornes incluses). On donne le formulaire HTML suivant :

```
<form name="inscription">
  <p>Nom : <input type="text" id="nom"></p>
  <p>Age : <input type="text" id="age"></p>
  <p><input type="button" value="Valider" onClick="verifier()"></p>
```

```
</form>
<p id="erreur"></p>
```

1. Écris la fonction JavaScript `verifier()` qui :
 - récupère le contenu des deux champs à l'aide de `document.getElementById()` ;
 - affiche dans le paragraphe `erreur` le message « Erreur : le nom est obligatoire. » si le champ `nom` est vide ;
 - affiche « Erreur : l'âge doit être un nombre. » si la conversion de l'âge avec `parseInt()` échoue ;
 - affiche « Erreur : l'âge doit être compris entre 17 et 25 ans. » si l'âge est hors de l'intervalle (on utilisera les opérateurs logiques `||` ou `&&`) ;
 - affiche « Inscription validée. Bienvenue ! » si tout est correct.
2. Explique le rôle de l'évènement `onClick` dans ce formulaire et cite deux autres événements JavaScript que l'on aurait pu utiliser pour améliorer l'ergonomie de ce formulaire, en précisant leur effet.
3. Le webmaster souhaite placer le script dans un fichier externe nommé `controle.js`. Écris la balise HTML exacte permettant de lier ce fichier à la page, et cite un avantage de cette méthode.

11 CORRIGÉS DES EXERCICES

11.1 Je vérifie mes connaissances

CORRIGÉ Exercice 1 — QCM : premiers pas en JavaScript

1. Réponse c. La balise standard d'insertion d'un script dans un document HTML est `<script>`. Les balises `<js>`, `<javascript>` et `<code>` n'existent pas dans le langage HTML.
2. Réponse b. `alert()` affiche une boîte de dialogue modale contenant un message. `prompt()` affiche une boîte de saisie (elle retourne une chaîne), `print()` n'existe pas en JavaScript côté client (pour la console, on utilise `console.log()`), et `echo()` appartient au langage PHP.
3. Réponse b. `==` est l'opérateur de comparaison d'égalité (égalité faible, avec conversion de type implicite). Attention : `=` est l'opérateur d'affectation, et `===` teste l'égalité stricte (valeur et type).
4. Réponse b. En JavaScript, les tableaux sont indexés à partir de 0 : `t[0]` vaut 5, `t[1]` vaut 9, `t[2]` vaut 2.

CORRIGÉ Exercice 2 — Vrai ou faux

1. Vrai. JavaScript est un langage interprété par le moteur JavaScript du navigateur de l'internaute (V8 pour Chrome, SpiderMonkey pour Firefox). Le code s'exécute donc sur la machine du client, et non sur le serveur web.
2. Faux. La fonction `prompt()` retourne toujours une chaîne de caractères (type `string`), même si l'utilisateur tape des chiffres. Sans conversion, l'opérateur `+` provoquerait une concaténation au lieu d'une addition. Il faut convertir la saisie avec `parseInt()`, `parseFloat()` ou `Number()`.
3. Vrai. Dans une boucle `do...while`, le test de la condition est effectué après l'exécution du corps de la boucle : le bloc d'instructions est donc exécuté au moins une fois, même si la condition est fausse dès le départ.
4. Faux. L'évènement `onLoad` se déclenche lorsque la page web a fini de se charger dans le navigateur. Le clic sur un bouton correspond à l'évènement `onClick`.

CORRIGÉ Exercice 3 — Définitions et question de cours

1. Script : un **script** est une suite d'instructions écrites dans un langage de script (ici JavaScript), insérée dans une page HTML (balise `<script>` ou fichier externe `.js`) et exécutée par le navigateur pour rendre la page dynamique et interactive.
Évènement : un **évènement** est une action détectable par le navigateur (clic de souris, survol d'un élément, saisie au clavier, fin de chargement de la page, modification d'un champ de formulaire...) susceptible de déclencher l'exécution d'une fonction JavaScript appelée gestionnaire d'évènement.
2. Avantages de JavaScript côté client (deux parmi) :
 - interactivité immédiate : le traitement est local, sans aller-retour vers le serveur, d'où une grande réactivité de la page ;
 - allègement du serveur : les contrôles et calculs simples sont réalisés sur le poste client, ce qui réduit la charge du serveur et la consommation de bande passante (atout précieux pour les connexions lentes).Limites (deux parmi) :
 - sécurité limitée : le code source est visible et modifiable par l'utilisateur ; un contrôle critique ne doit jamais reposer uniquement sur JavaScript ;
 - dépendance au navigateur : JavaScript peut être désactivé par l'utilisateur, et le comportement peut varier légèrement d'un navigateur à l'autre. De plus, l'accès au disque dur et aux bases de données est interdit au script (bac à sable).
3. `parseInt("7.9")` analyse la chaîne et s'arrête au premier caractère qui ne peut pas faire partie d'un entier (ici le point) : elle retourne l'entier 7.
`parseFloat("7.9")` analyse la chaîne comme un nombre décimal : elle retourne le nombre flottant 7,9 (noté 7.9 en JavaScript).

CORRIGÉ Exercice 4 — Évaluation d'expressions

On rappelle : `a = 10` (nombre), `b = "3"` (chaîne), `c = 4` (nombre).

1. `a + c = 10 + 4 = 14` (type `number`) : addition arithmétique entre deux nombres.
2. `a + b = "103"` (type `string`) : l'un des opérandes est une chaîne, donc l'opérateur `+` réalise une concaténation ; le nombre 10 est converti en "10".
3. `a + parseInt(b) = 10 + 3 = 13` (type `number`) : `parseInt("3")` convertit la chaîne en entier, l'addition redevient arithmétique.
4. `a % c = 10 mod 4 = 2` (type `number`) : reste de la division euclidienne de 10 par 4 ($10 = 4 \times 2 + 2$).
5. `(a > c) && (c < 5) = true` (type `boolean`) : `10 > 4` est vrai et `4 < 5` est vrai ; le ET logique

de deux valeurs vraies est vrai.

6. `parseFloat("12,5".replace(",", ".")) = 12,5 (type number)` : la méthode `replace` transforme la virgule française en point décimal, puis `parseFloat` convertit correctement la chaîne "12.5" en nombre.

Point de vigilance : la confusion entre addition et concaténation (questions 1 et 2) est l'erreur la plus fréquente au Probatoire : dès qu'un opérande de `+` est une chaîne, JavaScript concatène.

11.2 Je m'entraîne

CORRIGÉ Exercice 5 — Table de multiplication

1. Version avec la boucle `for` :

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Table de multiplication</title>
</head>
<body>
<script>
  var saisie = prompt("Entrez un nombre entier :");
  var n = parseInt(saisie);          // conversion en entier
  if (isNaN(n)){
    alert("Veuillez entrer un nombre entier valide.");
  } else {
    var msg = "Table de " + n + " :\n";
    for (var i = 1; i <= 10; i++){
      msg += n + " x " + i + " = " + (n * i) + "\n";
    }
    alert(msg);
  }
</script>
</body>
</html>
```

Vérification mentale : pour $n = 7$, la boucle s'exécute pour i de 1 à 10 ; à $i = 3$, la ligne produite est « 7 x 3 = 21 » ; à $i = 11$, la condition $i \leq 10$ devient fausse et la boucle s'arrête. Le message contient bien 10 lignes.

2. Version avec la boucle `while` (seul le bloc de boucle change) :

```
var msg = "Table de " + n + " :\n";
var i = 1;                      // initialisation AVANT la boucle
while (i <= 10) {                // condition testée avant chaque passage
  msg += n + " x " + i + " = " + (n * i) + "\n";
}
```

```
    i++;                // incrementation DANS le corps
  }
  alert(msg);
```

Point de vigilance : dans la version `while`, oublier l'instruction `i++` provoque une boucle infinie (la condition reste vraie indéfiniment et le navigateur se bloque). La boucle `for` regroupe initialisation, condition et incrémentation sur une seule ligne, ce qui limite ce risque.

CORRIGÉ Exercice 6 — Gestion des ventes d'une boutique

1. Fonction de calcul du total :

```
function totalVentes(t) {
  var somme = 0;           // accumulateur initialise a 0
  for (var i = 0; i < t.length; i++) {
    somme += t[i];         // somme = somme + t[i]
  }
  return somme;           // la fonction RETOURNE la somme
}
```

La propriété `t.length` donne le nombre d'éléments du tableau (ici 4) : la boucle parcourt donc les indices 0 à 3.

Vérification : $15000 + 8000 + 22000 + 12000 = 57000$. La fonction retourne bien 57000.

2. Fonction de recherche du maximum :

```
function plusGrandeVente(t) {
  var max = t[0];         // on suppose le 1er element maximal
  for (var i = 1; i < t.length; i++) {
    if (t[i] > max) {
      max = t[i];         // nouveau maximum trouve
    }
  }
  return max;
}
```

Trace d'exécution : `max = 15000`; $8000 < 15000$ (inchangé); $22000 > 15000$ donc `max = 22000`; $12000 < 22000$ (inchangé). La fonction retourne 22000.

3. Instruction d'affichage :

```
alert("Total : " + totalVentes(ventes) +
      " FCFA - Meilleure vente : " +
      plusGrandeVente(ventes) + " FCFA");
```

Le message affiché est : « Total : 57000 FCFA - Meilleure vente : 22000 FCFA ».

Point de vigilance : les deux fonctions utilisent `return` et non `alert()` : une fonction qui retourne une valeur peut être réutilisée dans un calcul ou une concaténation, ce qui est exactement ce que fait la question 3. Initialiser `max` à `t[0]` (et non à 0) garantit un résultat correct même si toutes les valeurs sont négatives.

CORRIGÉ Exercice 7 — Parité d'un nombre

Un entier est **pair** si le reste de sa division par 2 vaut 0, c'est-à-dire si `n % 2 === 0`.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Pair ou impair</title>
</head>
<body>
<script>
  var saisie = prompt("Entrez un nombre entier :");
  var n = parseInt(saisie);      // conversion obligatoire
  if (isNaN(n)){
    alert("Saisie invalide : ce n'est pas un nombre entier.");
  } else {
    if (n % 2 === 0){
      alert("Le nombre est pair");
    } else {
      alert("Le nombre est impair");
    }
  }
</script>
</body>
</html>
```

Vérifications : pour $n = 14$, $14 \bmod 2 = 0$ donc « pair » ; pour $n = 7$, $7 \bmod 2 = 1$ donc « impair » ; pour $n = 0$, $0 \bmod 2 = 0$ donc « pair » (0 est bien un nombre pair).

Point de vigilance : le test s'écrit `n % 2 === 0` avec l'opérateur de comparaison `===` (ou `==`), jamais `n % 2 = 0` qui serait une affectation interdite.

CORRIGÉ Exercice 8 — Premiers évènements

La page complète, avec les trois gestionnaires d'évènements ajoutés :

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Accueil</title>
```

```
</head>
<body onload="alert('Bienvenue sur notre site !')">
  <p id="message"
    onmouseover="document.getElementById('message').textContent = 'Bravo, vous m\'avez survolé !'">
    Survolez-moi !
  </p>
  <button onclick="alert('Merci pour votre clic')">Cliquez-moi</button>
</body>
</html>
```

Explications :

1. L'évènement `onload` se place sur la balise `<body>` : il se déclenche quand la page est entièrement chargée.
2. L'évènement `onmouseover` se déclenche quand le pointeur de la souris entre dans la zone du paragraphe. `document.getElementById('message')` désigne l'élément dont l'attribut `id` vaut `message`, et sa propriété `textContent` permet d'en modifier le texte.
3. L'évènement `onclick` du bouton déclenche la boîte de dialogue.

Point de vigilance : les attributs HTML sont délimités par des guillemets doubles ; les chaînes JavaScript qu'ils contiennent utilisent donc des guillemets simples, et l'apostrophe intérieure doit être protégée par un antislash (`\`). Une confusion entre guillemets simples et doubles est une cause fréquente de script muet (aucune erreur visible, mais rien ne se passe).

11.3 Je me prépare au Probatoire

CORRIGÉ Exercice 9 — Chasse aux erreurs

1. Les cinq erreurs et leurs corrections :

- (a) `var age = saisie;` — erreur de conversion. `prompt()` retourne une chaîne de caractères ; les comparaisons et calculs ultérieurs seront faussés. Correction : `var age = parseInt(saisie);`
- (b) `if (age = 18)` — erreur de logique (et de syntaxe algorithmique). `=` est une affectation : la variable `age` reçoit 18 et la condition est toujours vraie. Correction : `if (age === 18)` (égalité stricte) ou `if (age == 18)`.
- (c) `ages[3]` — erreur d'accès au tableau. Le tableau `ages` contient 3 éléments, d'indices 0, 1 et 2 ; l'indice 3 est hors limites et vaut `undefined`. Pour afficher le premier âge : correction : `ages[0]` (valeur 15).
- (d) `document.getElementById(resultat)` — erreur d'accès à un élément HTML. Sans guillemets, `resultat` est interprété comme une variable JavaScript inexistante ; de

plus aucun élément de la page ne porte l'identifiant `resultat`. Correction : écrire `document.getElementById("resultat")` et ajouter dans le corps de la page un élément `<p id="resultat"></p>`, ou supprimer cette ligne si elle est inutile.

- (e) La balise fermante `<script>` en fin de script — erreur de syntaxe HTML. Une balise fermante porte une barre oblique. Correction : `</script>`.
2. Pourquoi la comparaison reste risquée sans conversion : tant que `age` est une chaîne (par exemple "20"), JavaScript effectue une conversion implicite lors de la comparaison avec un nombre. Cela peut donner un résultat juste par chance pour "20", mais toute saisie non numérique (par exemple "abc") est convertie en `NaN`, et toute comparaison avec `NaN` est fausse : un élève majeur serait alors déclaré mineur. Correction définitive : convertir dès la saisie, `var age = parseInt(saisie);`, et contrôler la validité avec `isNaN(age)`.
3. Affichage pour la saisie 20 (script corrigé) : `age` vaut 20. Le test `age === 18` est faux : pas de premier message. Le test `age >= 18` est vrai : la boîte de dialogue affiche « Vous êtes majeur. Premier age du tableau : 15 ». Le bloc `else` n'est pas exécuté.

Barème indicatif (10 pts) : identification et correction des 5 erreurs : $5 \times 1 \text{ pt} = 5 \text{ pts}$; explication du problème de conversion : 2 pts ; correction définitive proposée : 1 pt ; affichage exact pour la saisie 20 (avec la valeur 15 du tableau) : 2 pts.

Points de vigilance : le correcteur attend pour chaque erreur la nature (conversion, logique, indice de tableau, accès DOM, syntaxe HTML) et la correction exacte, pas seulement la ligne fautive. Pour la question 3, la valeur affichée doit être `ages[0] = 15`, pas 3 ni `undefined`.

CORRIGÉ Exercice 10 — Mentions au Probatoire TI

1. La fonction `mention(n)` :

```
function mention(n) {  
  if (n < 10) {  
    return "Ajourne";  
  } else if (n < 12) { // ici n >= 10 est déjà garanti  
    return "Passable";  
  } else if (n < 16) { // ici n >= 12 est déjà garanti  
    return "Bien";  
  } else { // n >= 16  
    return "Tres bien";  
  }  
}
```

On pouvait aussi écrire les conditions en extension avec des opérateurs logiques, par exemple `else if (n >= 10 && n < 12)`.

2. Le script principal :

```

var saisie = prompt("Entrez la note sur 20 :");
var note = parseFloat(saisie); // note éventuellement décimale
if (isNaN(note)){
    alert("Erreur : veuillez entrer un nombre valide.");
} else {
    var m = mention(note);
    alert("Note : " + note + " - Mention : " + m);
}

```

Pour la saisie 13.5, on obtient : « Note : 13.5 – Mention : Bien » ($12 \leq 13,5 < 16$).

3. Saisie et affichage des 5 notes avec un tableau créé par `new Array()` :

```

var notes = new Array(5); // tableau de 5 cases
// Remplissage du tableau
for (var i = 0; i < 5; i++){
    var s = prompt("Note de l'élève " + (i + 1) + " :");
    notes[i] = parseFloat(s);
}
// Affichage des mentions
for (var i = 0; i < 5; i++){
    if (isNaN(notes[i])){
        alert("Élève " + (i + 1) + " : note invalide");
    } else {
        alert("Élève " + (i + 1) + " : Note " + notes[i] +
            " - Mention : " + mention(notes[i]));
    }
}

```

4. Cas de la saisie "abc" : `parseFloat("abc")` retourne NaN. Or toute comparaison impliquant NaN est fautive : $\text{NaN} < 10$, $\text{NaN} < 12$ et $\text{NaN} < 16$ sont toutes fausses. L'exécution tombe donc dans le `else final` et l'élève obtient abusivement la mention « Très bien ».
- Test à ajouter en tête de la fonction : `if (isNaN(n)) { return "Note invalide"; }`.

Barème indicatif (10 pts) : fonction `mention` correcte (structure `if...else if...else`, quatre cas, `return`) : 3 pts ; script principal (`prompt`, `parseFloat`, appel, affichage) : 2,5 pts ; tableau `new Array(5)` et deux boucles : 3 pts ; réponse NaN → « Très bien » justifiée + test `isNaN` : 1,5 pt.

Points de vigilance : le barème impose des structures alternatives emboîtées ; l'ordre des tests doit respecter les intervalles croissants. Citer explicitement que `parseFloat` est utilisé (et non `parseInt`) car une note peut être décimale (13,5).

CORRIGÉ

Exercice 11 — Formulaire d'inscription à un concours

1. La fonction `verifier()` :

```

function verifier() {
    // Récupération des éléments HTML par leur identifiant
}

```

```
var nom = document.getElementById("nom").value;
var ageSaisi = document.getElementById("age").value;
var zone = document.getElementById("erreur");

// Controle 1 : le nom est obligatoire
if (nom == ""){
    zone.textContent = "Erreur : le nom est obligatoire.";
    return; // on arrete la fonction
}

// Controle 2 : l'age doit etre un nombre entier
var age = parseInt(ageSaisi);
if (isNaN(age)){
    zone.textContent = "Erreur : l'age doit etre un nombre.";
    return;
}

// Controle 3 : l'age doit etre dans l'intervalle [17 ; 25]
if (age < 17 || age > 25){
    zone.textContent =
        "Erreur : l'age doit etre compris entre 17 et 25 ans.";
    return;
}

// Tous les controles sont passes
zone.textContent = "Inscription validee. Bienvenue !";
}
```

La condition « hors intervalle » s'écrit `age < 17 || age > 25` (opérateur OU); de façon équivalente, la condition de validité est `age >= 17 && age <= 25` (opérateur ET). Vérification : âge 17 → accepté (borne incluse); âge 25 → accepté; âge 16 ou 26 → refusé.

2. Rôle de `onClick` : placé sur le bouton « Valider », l'évènement `onClick` déclenche l'exécution de la fonction `verifier()` au moment où l'utilisateur clique : c'est le lien entre l'action de l'utilisateur et le script de contrôle.

Deux autres évènements pour améliorer l'ergonomie :

- `onChange` sur le champ âge : le contrôle se déclenche dès que l'utilisateur quitte le champ après l'avoir modifié, sans attendre le clic sur « Valider »;
- `onFocus` sur un champ : il se déclenche quand le champ reçoit le curseur; on peut par exemple effacer le message d'erreur ou colorer le champ pour guider la saisie.

3. Liaison du fichier externe :

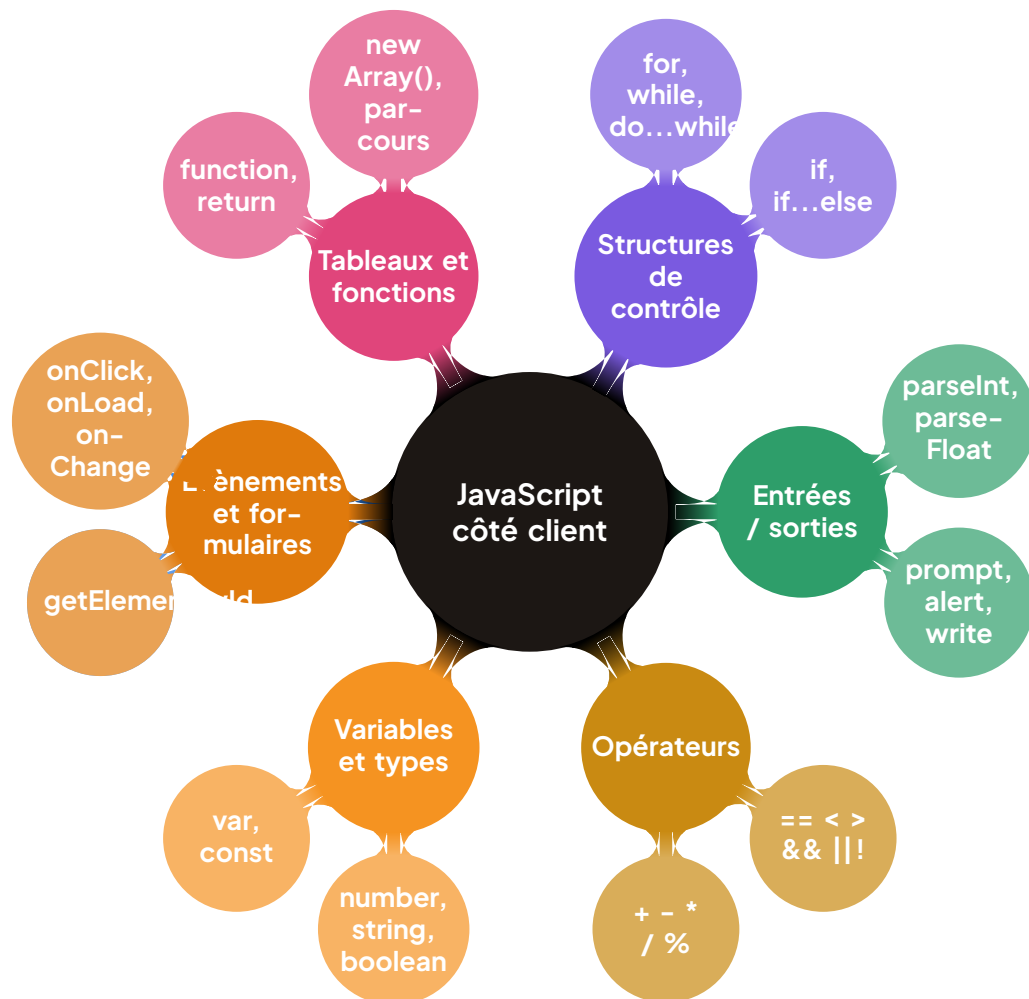
```
<script src="controle.js"></script>
```

Avantage : le script est séparé du document HTML, ce qui rend la page plus lisible, permet de réutiliser le même fichier `controle.js` dans plusieurs pages, et facilite la maintenance (une seule modification profite à tout le site).

Barème indicatif (10 pts) : récupération correcte des champs avec `getElementById(...).value` : 2 pts; contrôle du nom vide : 1,5 pt; conversion `parseInt` + test `isNaN` : 2 pts; test d'intervalle avec opérateur logique : 1,5 pt; message de succès : 0,5 pt; rôle de `onClick` et deux évènements justifiés : 1,5 pt; balise `src` et avantage : 1 pt.

Points de vigilance : ne pas oublier la propriété `.value` après `getElementById(...)` (sans elle, on récupère l'élément et non son contenu); les bornes 17 et 25 sont incluses, d'où les comparaisons strictes `< 17` et `> 25` pour le rejet; l'identifiant passé à `getElementById` doit être entre guillemets.

12 FICHE BILAN



Carte mentale de la leçon : les sept blocs de compétences JavaScript côté client.

FICHE BILAN L'essentiel à connaître pour l'examen

1. Définitions clés

- **Script** : petit programme inséré dans une page web et exécuté par le navigateur (côté client), sans compilation.
- **JavaScript** : langage de script interprété, non typé à la déclaration, qui rend les pages HTML dynamiques (réaction aux actions de l'utilisateur).
- **Évènement** : action détectée par le navigateur (clic, survol, chargement, saisie...) à laquelle on associe du code JavaScript.

- **Variable** : zone mémoire nommée dont la valeur peut changer ; **constante** : zone mémoire dont la valeur est fixée une fois pour toutes.

2. Syntaxes à connaître par cœur

Notion	Syntaxe JavaScript
Insertion dans HTML	<code><script type="text/javascript"> ... </script></code> (dans <code><head></code> ou <code><body></code>); fichier externe : <code><script src="fichier.js"></script></code>
Déclaration	<code>var age = 17 ; const PI = 3.14 ;</code>
Types de base	number (entier ou réel), string (chaîne), boolean (true/false)
Entrée / sortie	<code>prompt("message"),</code> <code>alert("message"),</code> <code>document.write("texte")</code>
Conversions	<code>parseInt("25") → 25</code> ; <code>parseFloat("2,5") → 2.5</code>
Alternative	<code>if (cond) { ... } else { ... }</code>
Boucles	<code>for (i=0; i<n; i++), while (cond), do { ... } while (cond);</code>
Tableau	<code>var t = new Array(3);</code> ou <code>var t = [10, 20, 30];</code> ; accès : <code>t[0]</code> ; taille : <code>t.length</code>
Fonction	<code>function nom(p1, p2) { ... return r; }</code>
Évènements	<code>onClick, onMouseOver, onChange, onFocus, onLoad</code> (attributs des balises HTML)
Accès aux éléments	<code>document.getElementById("id").value</code>

3. Opérateurs

- Arithmétiques : `+` `-` `*` `/` `%` (reste de la division entière).
- Comparaison : `==`, `!=`, `<`, `>`, `<=`, `>=`.
- Logiques : `&&` (ET), `||` (OU), `!` (NON).
- Assignment : `=`, `+=`, `-=`, `++`, `--`.
- Concaténation de chaînes : `+` (ex. `"Bonjour " + nom`).

4. Méthodes express

- Contrôler un formulaire : récupérer la valeur avec `getElementById(...).value`, tester avec `if`, afficher un message avec `alert`, renvoyer `false` pour bloquer l'envoi.
- Parcourir un tableau : `for (i=0; i<t.length; i++)` puis tester chaque `t[i]` pour une recherche.
- Saisie numérique : toujours appliquer `parseInt` ou `parseFloat` au résultat de `prompt`, qui renvoie une chaîne.

À RETENIR

Checklist avant le Bac

- ☐ Je sais définir un script et un évènement, et citer deux avantages et une limite du JavaScript.
- ☐ Je sais insérer un script dans l'en-tête, dans le corps et via un fichier externe.
- ☐ Je sais déclarer une variable (`var`) et une constante (`const`) et citer les trois types de base.
- ☐ Je maîtrise les opérateurs arithmétiques, de comparaison, logiques, d'assignation et la concaténation avec `+`.
- ☐ Je sais convertir une saisie avec `parseInt()` et `parseFloat()`.
- ☐ Je sais lire une donnée (`prompt`) et afficher un résultat (`alert`, `document.write`).
- ☐ Je sais écrire un `if...else` et choisir entre `for`, `while` et `do...while`.
- ☐ Je sais créer un tableau avec `new Array()`, accéder à un élément et le parcourir pour rechercher une valeur.
- ☐ Je sais écrire une fonction avec ou sans valeur de retour (`return`).
- ☐ Je sais associer `onClick`, `onMouseOver`, `onChange`, `onFocus`, `onLoad` à des balises HTML.
- ☐ Je sais contrôler la saisie d'un formulaire avec `document.getElementById(...).value`.
- ☐ Je n'oublie pas les points-virgules, les accolades et la différence entre `=` (affectation) et `==` (comparaison).



Signé • OnBuch+

Ludovic A.

CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+